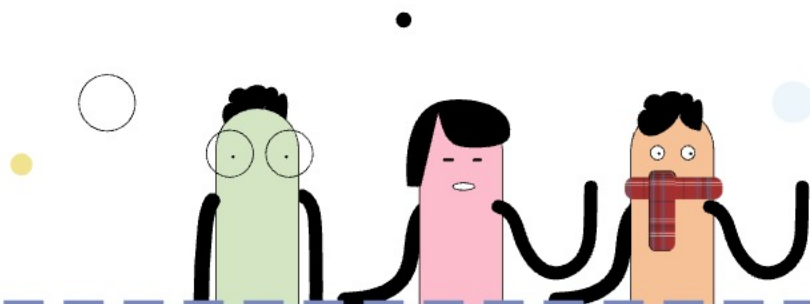
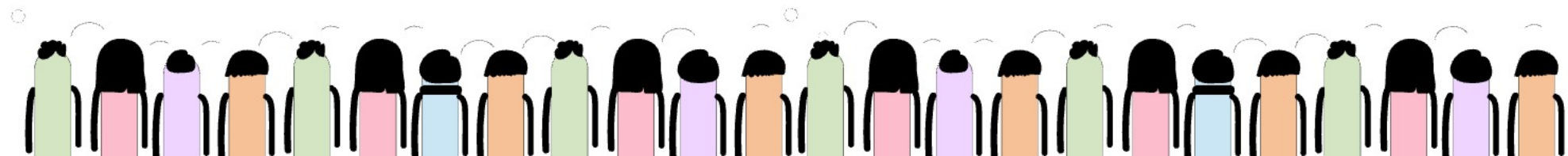
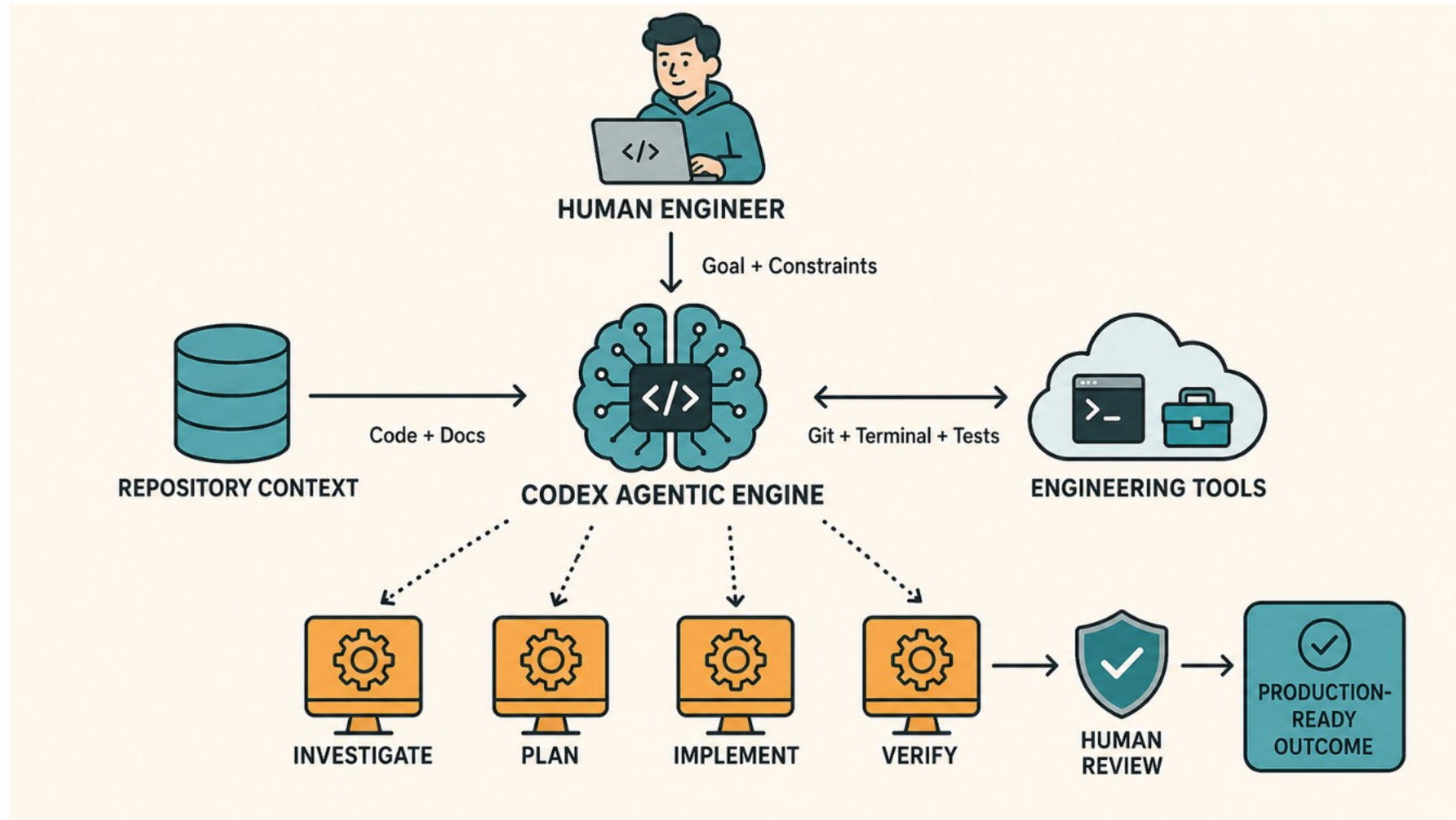




Codex

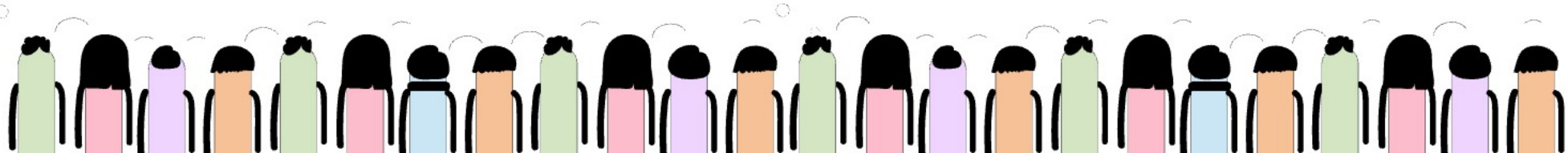
Internals

Codex



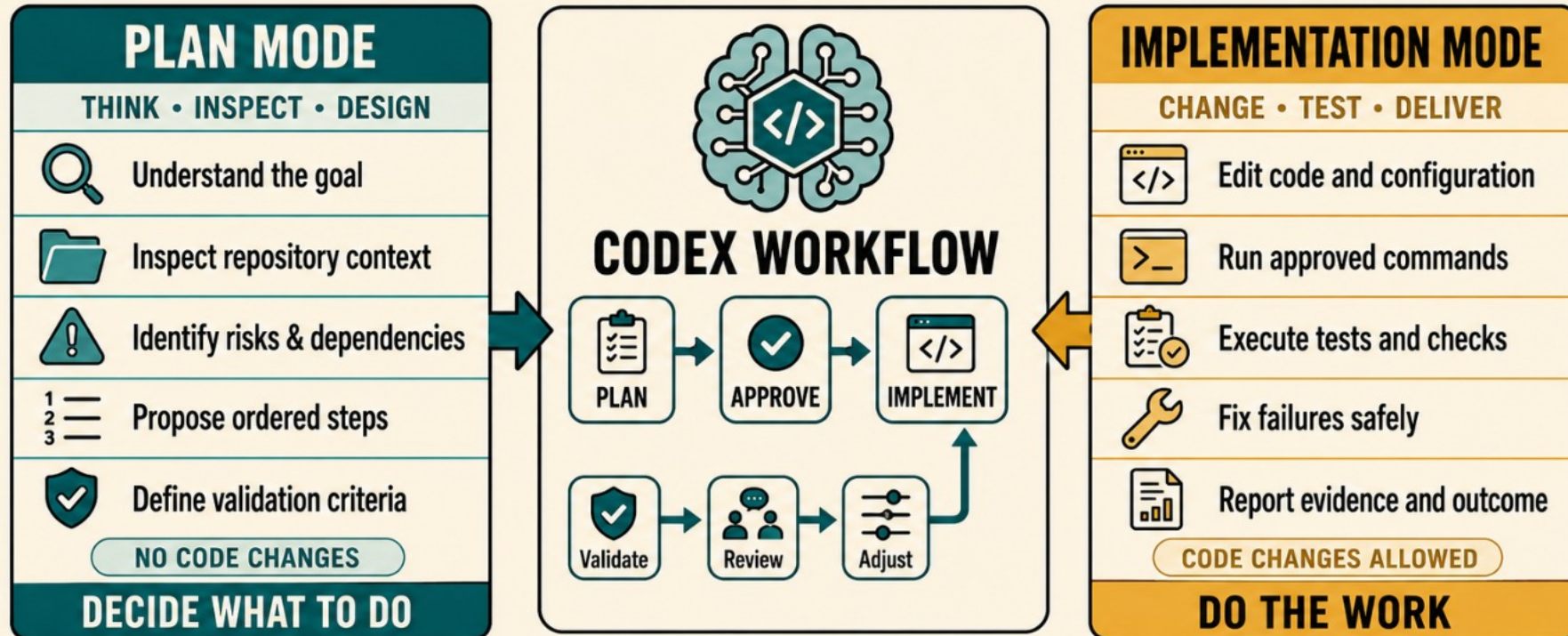
Prompt

CODEX AGENTIC SOFTWARE ENGINEERING WORKFLOW



Mode

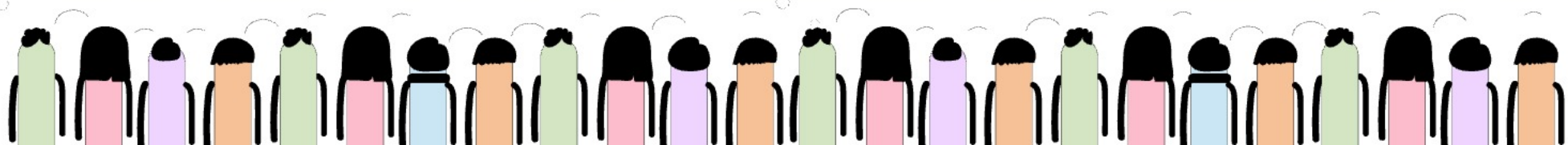
PLAN MODE vs IMPLEMENTATION MODE



BEST RESULTS = CLEAR PLAN + VERIFIED IMPLEMENTATION

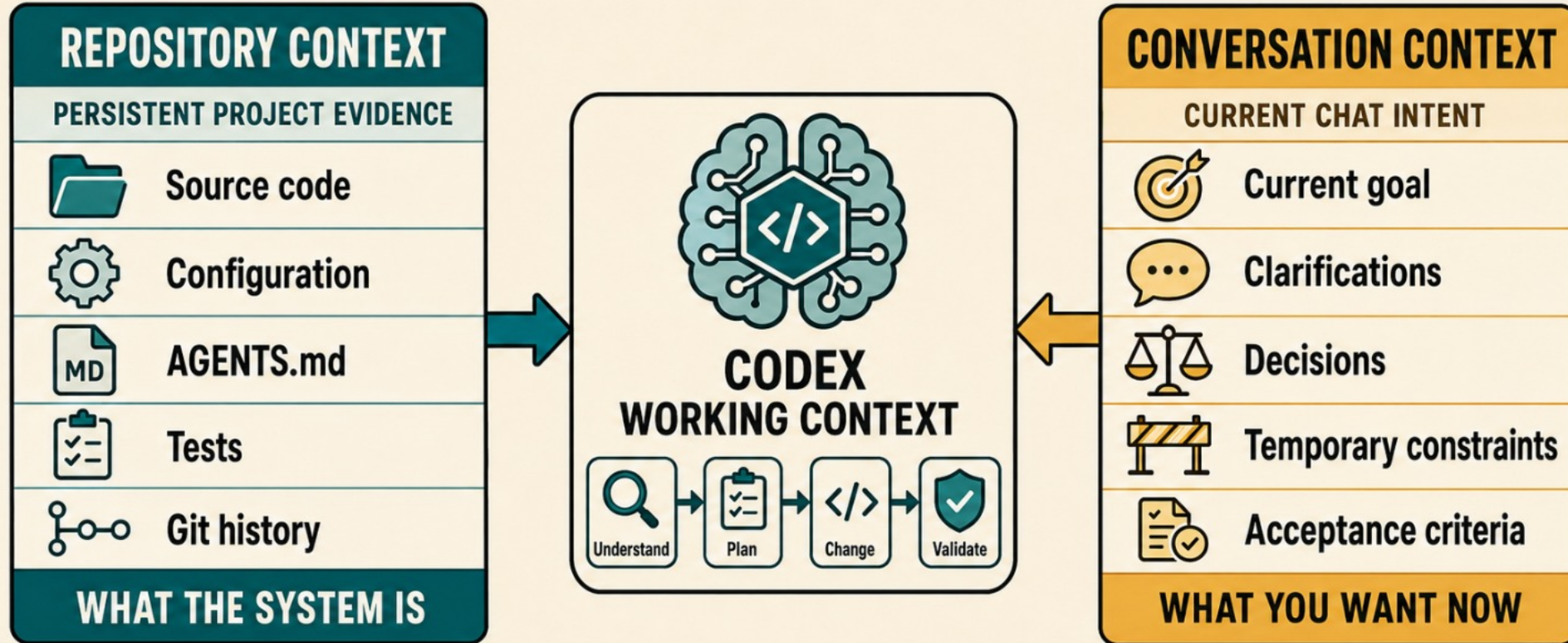


REAL SCENARIO: Payment API timeout → inspect traces → plan retry + circuit breaker → implement → load test → report



Context

REPOSITORY CONTEXT vs CONVERSATION CONTEXT



BEST RESULTS = CORRECT CODEBASE EVIDENCE + CLEAR CONVERSATION INTENT

Keep one chat focused on one coherent outcome

Codex VS CLAUDE

CLAUDE CODE vs CODEX

CHOOSE BY TASK EVIDENCE — NOT BRAND PREFERENCE

CLAUDE CODE

NATURAL FIT WHEN

	Anthropic stack or Claude API
	CLAUDE.md + hooks already adopted
	Agent SDK in Python / TypeScript
	Claude-centered terminal & IDE workflow

ECOSYSTEM FIT



BOTH ARE STRONG AT

	Repository understanding
	Planning + implementation
	Commands + tests
	MCP + external tools
	CI + agent workflows

NO UNIVERSAL WINNER

CODEX

NATURAL FIT WHEN

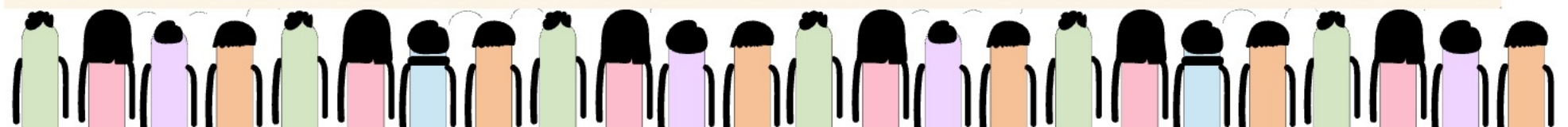
	ChatGPT + Codex stack
	AGENTS.md + skills + plugins
	Local-to-cloud task handoff
	codex exec + built-in review workflow

ECOSYSTEM FIT



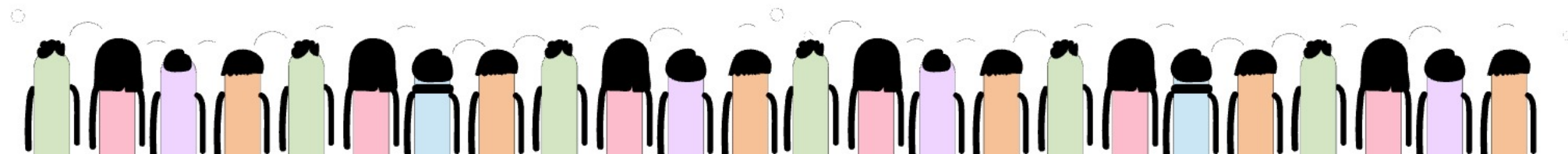
BEST TOOL = VERIFIED QUALITY + LOW REVIEW EFFORT + ACCEPTABLE TIME, COST & RISK

 CLAUDE.md + hook pipeline → CLAUDE CODE	 AGENTS.md + Codex cloud workflow → CODEX	 NEUTRAL REPOSITORY → BENCHMARK BOTH
---	---	---





Codex



Codex Prompting

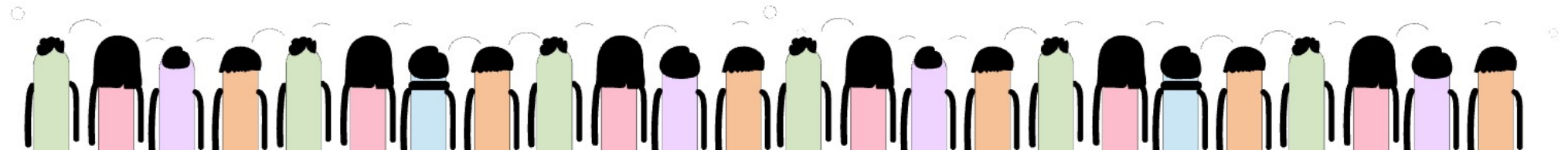


1 Current prompt

Purpose: the task to complete now.

Contains the objective, constraints, required process, approval gates, and final output.

Example: prevent duplicate jobs and test concurrent submissions.



Codex Prompting



2



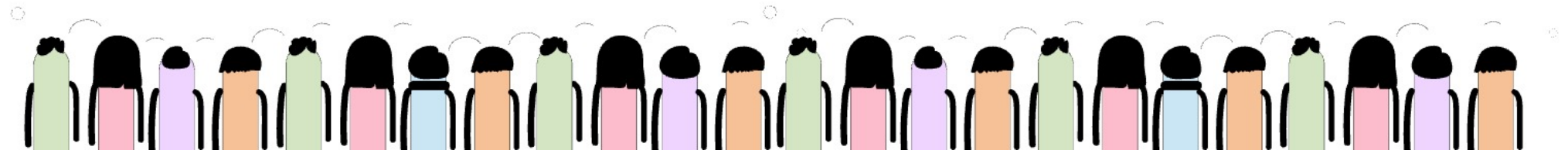
Applicable

AGENTS.md

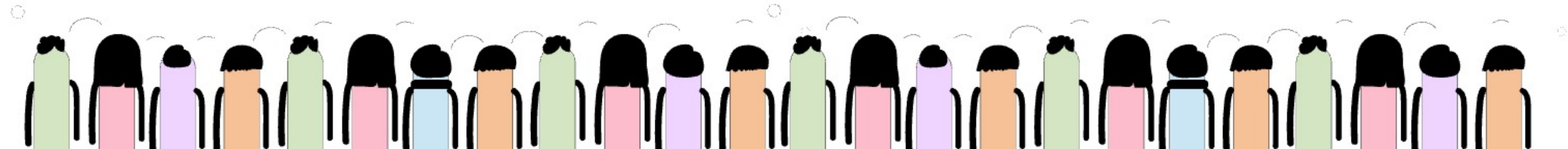
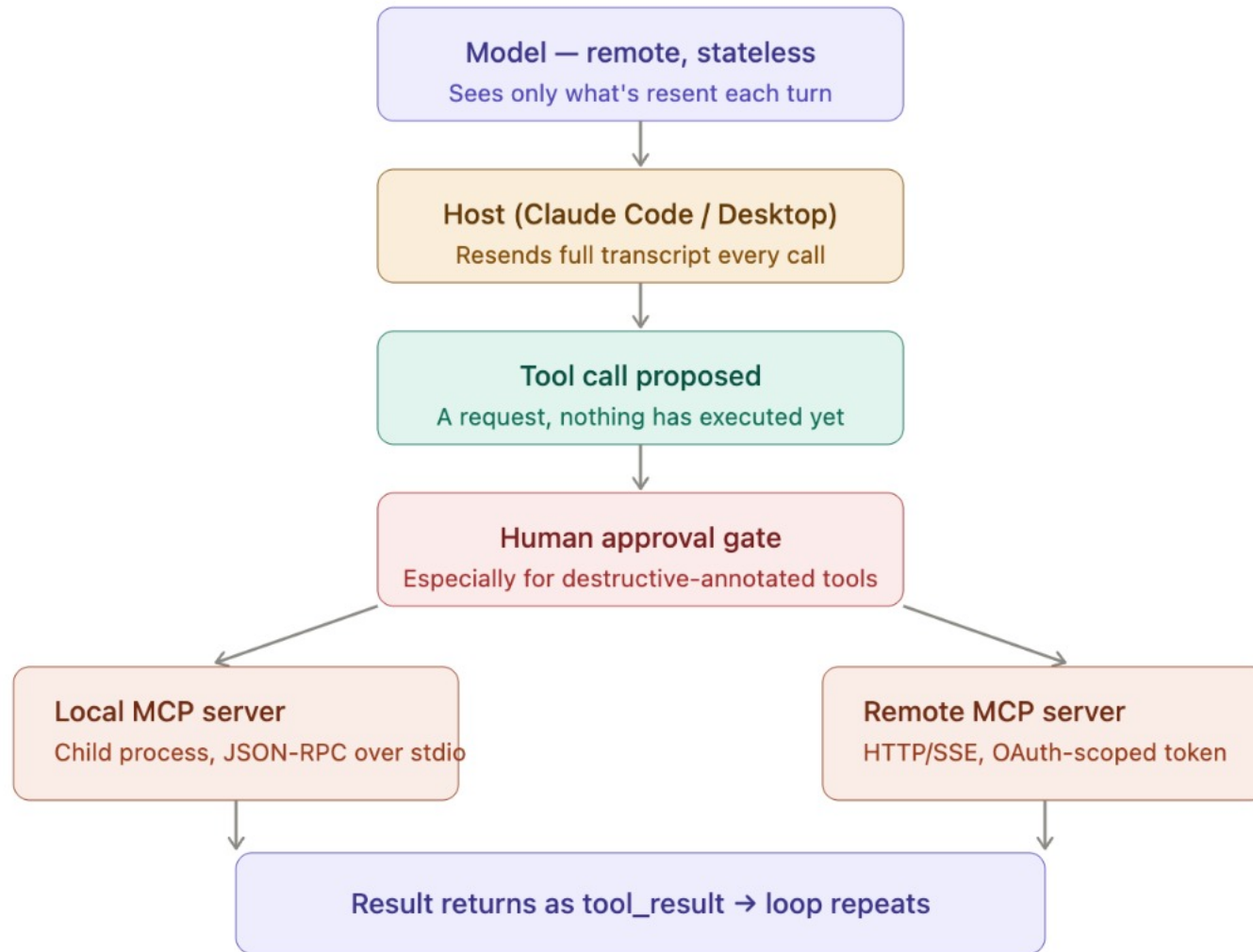
Purpose: durable repository guidance.

Supplies project layout, conventions, documented commands, architecture rules, and definition of done.

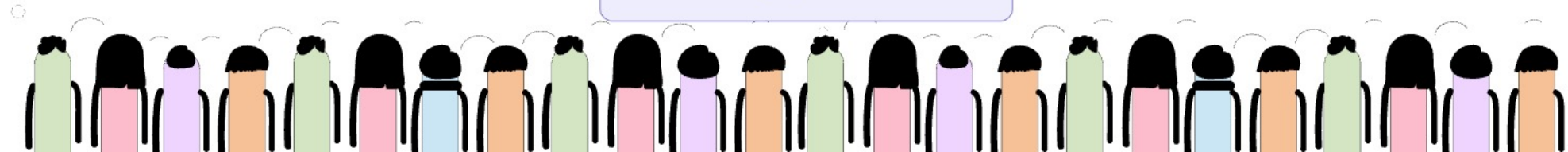
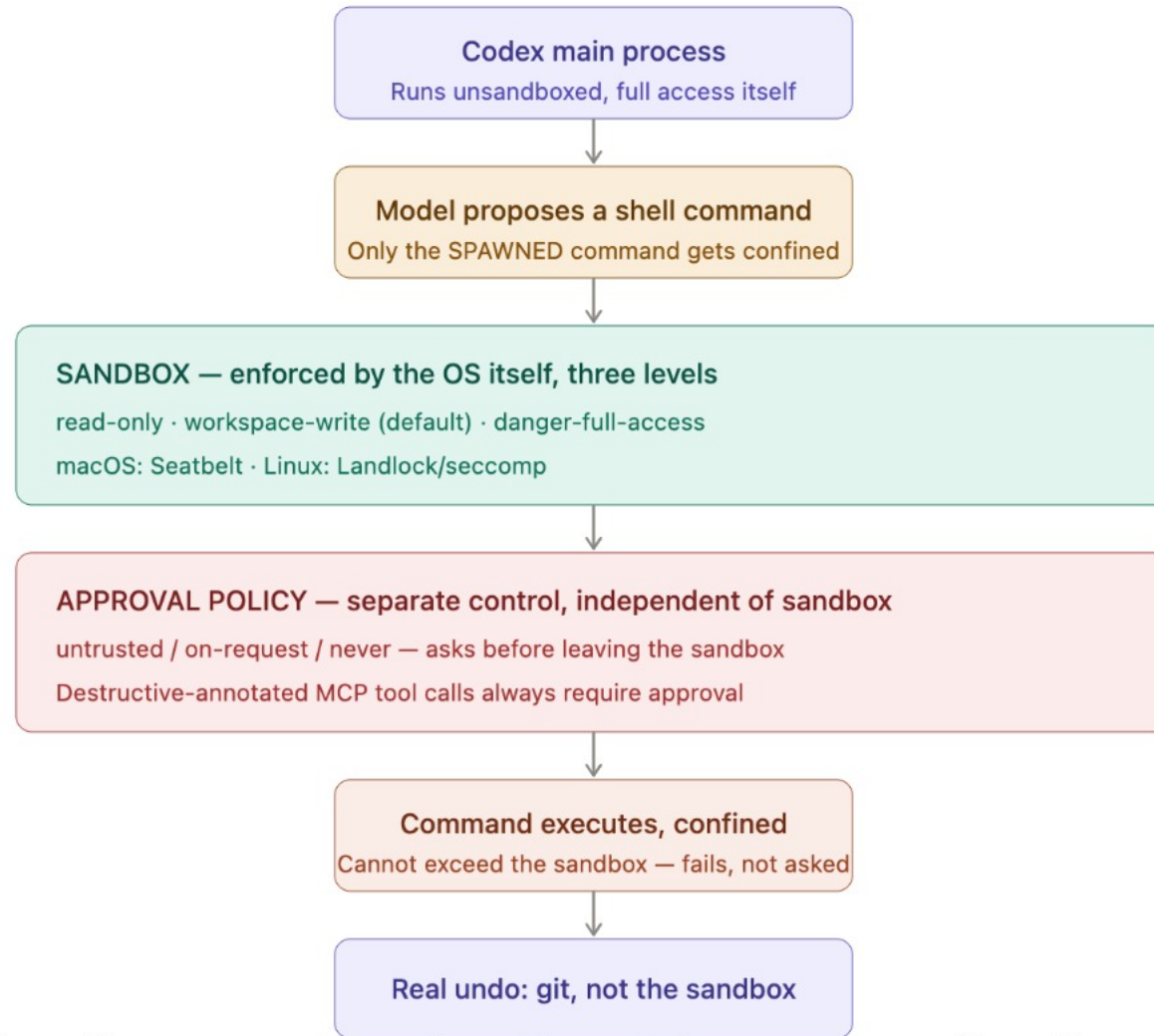
Codex loads guidance from the repository root down to the active directory.



Claude Ai thinking



Codex Ai thinking



Codex Skillset

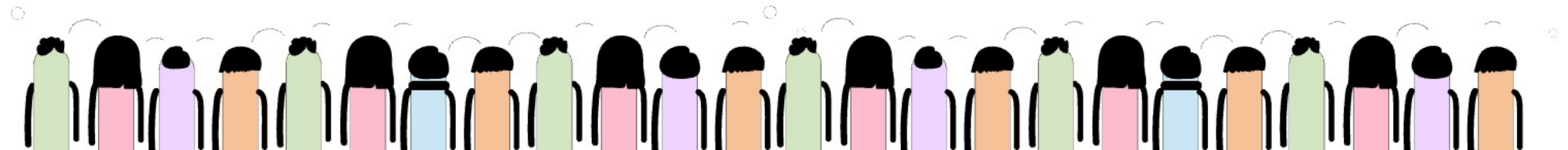


3 Reusable skill

Purpose: a repeatable specialist workflow.

May contribute detailed instructions, scripts, references, and templates for a recurring task.

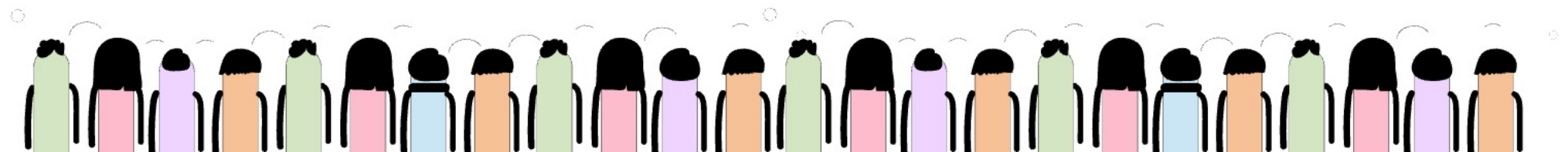
A skill is optional: it must be available and either explicitly invoked or implicitly matched.



Codex Slash Command

> /

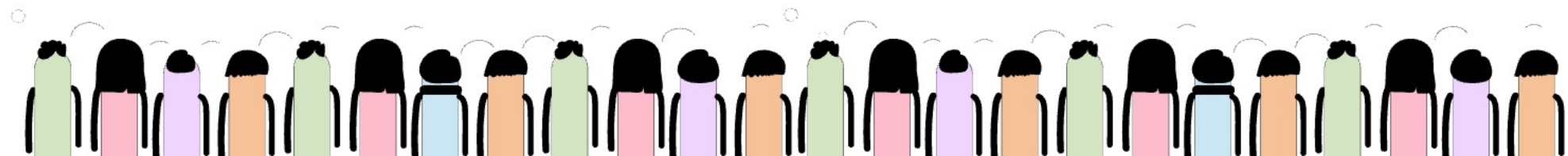
/model	choose what model and reasoning effort to use
/fast	1.5x speed, increased usage
/ide	include current selection, open files, and other context from your IDE
/permissions	choose what Codex is allowed to do
/keymap	remap TUI shortcuts
/vim	toggle Vim mode for the composer
/experimental	toggle experimental features
/approve	approve one retry of a recent auto-review denial



Codex Slash Command



Command	What it does	Importance	When to use	
<code>/permissions</code>	Controls what Codex may read, modify, execute and access	★★★★★	Before investigation, coding or security review	
<code>/model</code>	Selects the AI model and reasoning effort	★★★★★	Choose speed for simple work or deeper reasoning for architecture/debugging	
<code>/ide</code>	Adds your selected code, open files and IDE context	★★★★★	When asking about code currently open in VS Code/IDE	



Codex Slash Command

Codex \$ menu: choose the right capability

Type \$, select a capability, press Enter, then write a focused task.

>_ What does the task need?

Instructions, a repeatable workflow, or an external system?



Normal prompt

One-time coding or explanation task

Review src/payment.py for bugs

[Skill]

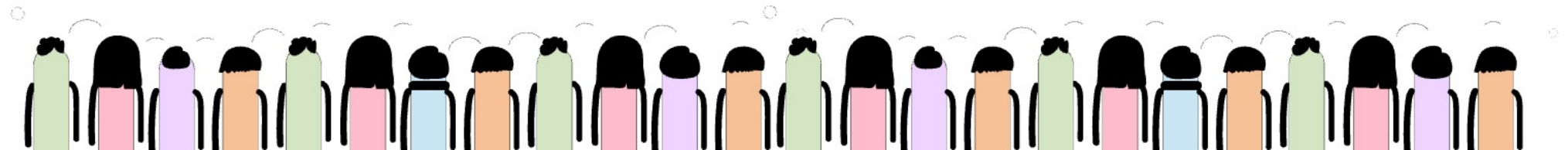
Reusable instructions, workflow, templates, examples or scripts

Documents · PDF · Image Gen · OpenAI Docs · Browser

[App]

Connected service, live session, external information or action

Document Control · Excel Live Control · Hotline



Codex Slash Command



What matters most

1. Correct selection

Use a Skill for how to work; use an App for where data or action lives.

2. Clear task scope

State the goal, input, constraints, expected output and validation.

3. Permission safety

Review access before an App changes external data or a Skill edits files.

4. Explicit vs automatic

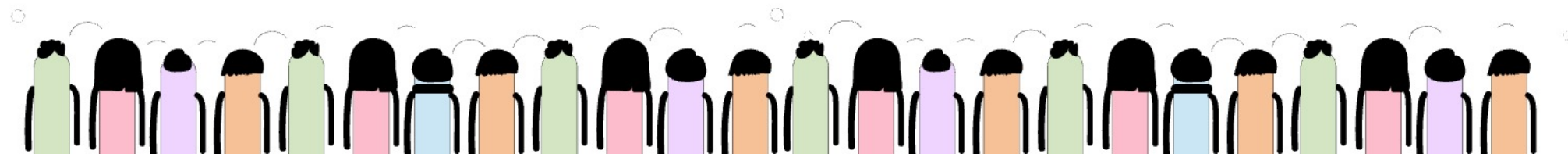
Select with `$` when you require one capability; Codex may also match Skills automatically.

5. Inspect the result

Review diffs, generated files, sources, commands and external actions.

6. Reuse proven work

Create a focused Skill when the same process and output repeat.

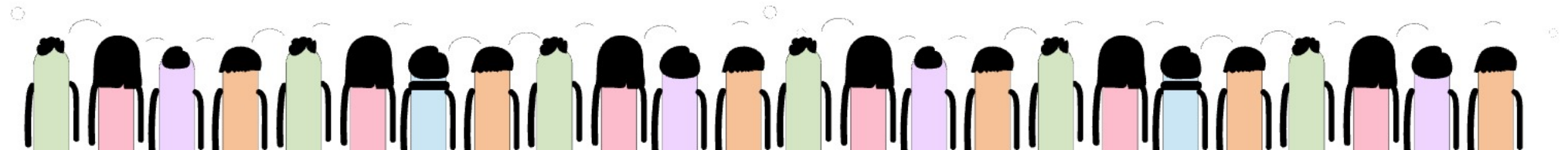


Codex Slash Command



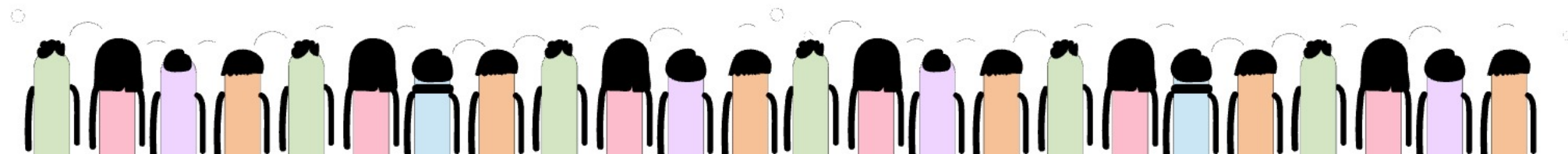
Five practice exercises

- 1 `$OpenAI Docs` Explain Codex Skills, Apps and Plugins with one example each.
- 2 `$Documents` Create a one-page training guide from these notes.
- 3 `$PDF` Read this architecture PDF and list the five production risks.
- 4 `$Image Gen` Create a clean system architecture illustration for training.
- 5 `$Excel Live Control` Inspect the open workbook and summarize failed test cases.

[Read](#)[Create](#)[Analyze](#)[Visual](#)[External action](#)

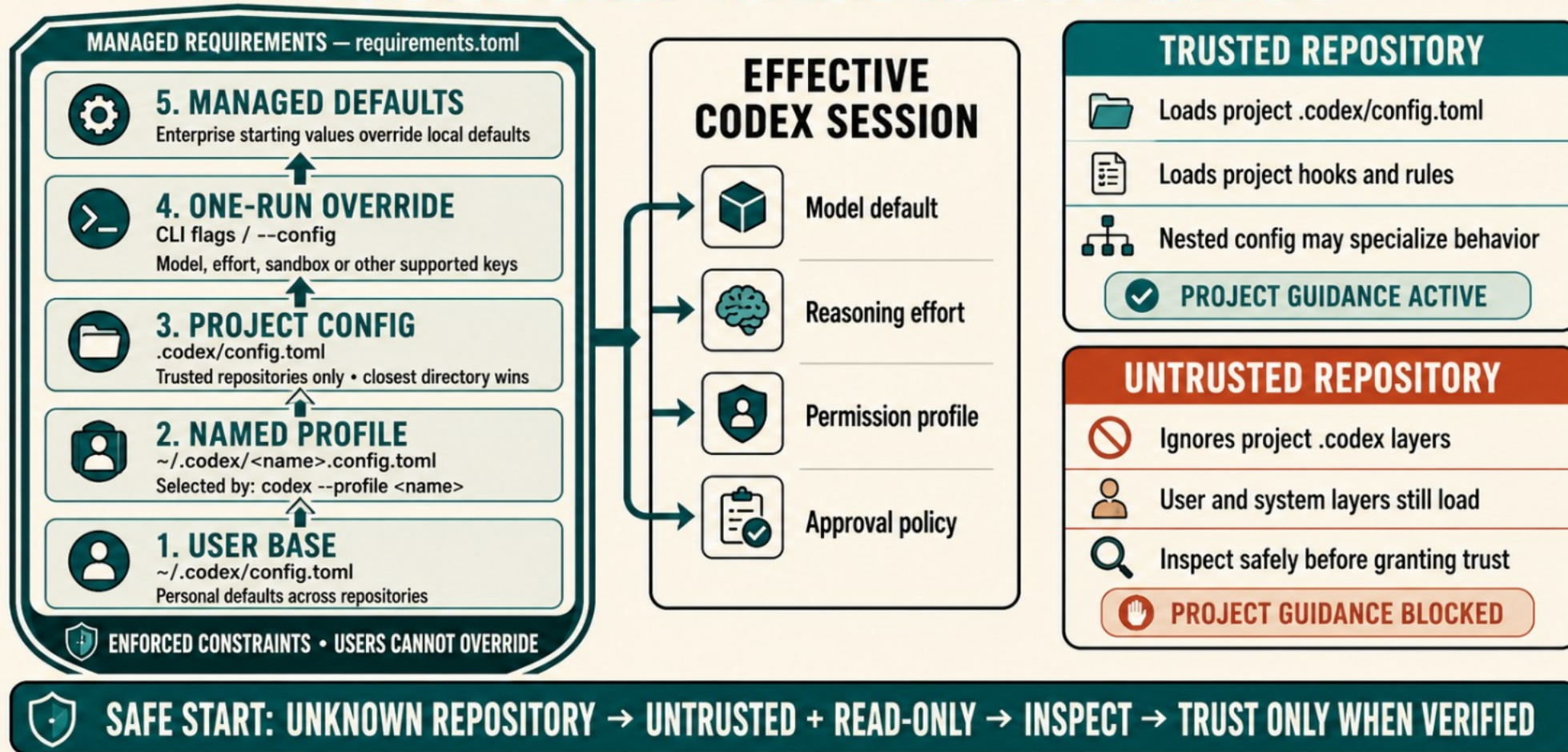


Codex

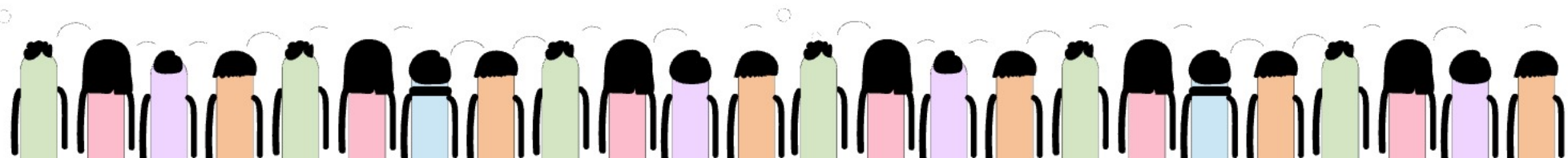


CODEX CONFIGURATION & REPOSITORY TRUST

WHO SETS THE DEFAULTS — AND WHICH PROJECT SETTINGS MAY LOAD



① Higher reasoning effort may improve complex work, but increases time and token usage.



CODEX PERMISSIONS, APPROVALS & ENTERPRISE GUARDRAILS

ACCESS BOUNDARY + DECISION POLICY + ORGANIZATION CONTROL

READ-ONLY

 **LOWEST CHANGE RISK**

-  Inspect files
-  No workspace edits
-  Commands or escalation need approval
-  Best for unknown repositories

WORKSPACE-WRITE

 **BALANCED DEFAULT**

-  Read files
-  Write inside workspace roots
-  Protected .git and .codex paths
-  Network requires explicit access

DANGER-FULL-ACCESS

 **HIGHEST RISK**

-  No local sandbox boundary
-  Broad filesystem access
-  Broad network exposure
-  Trusted, isolated environments only

APPROVAL POLICY — WHEN MUST AN ACTION STOP?



ON-REQUEST

Ask when escalation is needed



UNTRUSTED

Safe reads run; risky commands ask



NEVER

No prompts; sandbox still limits actions



GRANULAR

Prompt selected categories; reject others



Sandbox



Rules



GRANULAR CATEGORIES

MCP elicitations



Permission requests



Skill scripts

WHO REVIEWS?



HUMAN

— approvals_reviewer = user
User explicitly approves or denies



AUTO-REVIEW

— approvals_reviewer = auto_review
Reviewer agent evaluates eligible risk; failures close safely



Auto-review may use additional model calls

ENTERPRISE REQUIREMENTS



requirements.toml cannot be overridden



Allow approved permission profiles



Restrict policies and reviewers



Control network, MCP, hooks and features



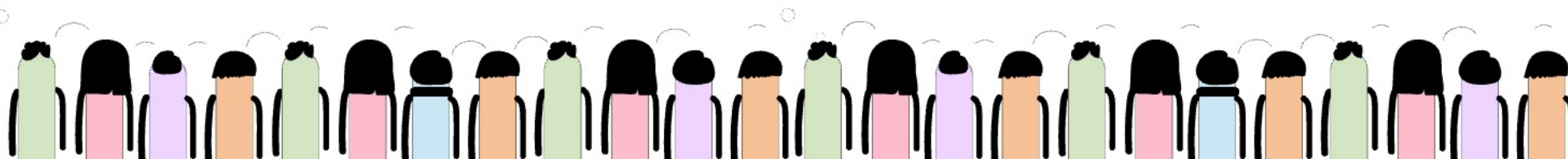
Managed defaults reset at startup



Runtime policy is separate from workspace RBAC



SECURE PATTERN = LEAST PRIVILEGE + TRUSTED REPOSITORY + NETWORK ALLOWLIST + APPROVALS + AUDITABLE POLICY



LEAST-PRIVILEGE AGENT ARCHITECTURE

GIVE EACH AGENT ONLY THE ACCESS REQUIRED FOR ITS ROLE

1. INVESTIGATION AGENT

READ-ONLY

- Inspect source code
- Read logs and test results
- Trace failures
- Produce findings and a plan

NO EDITS • NO PRODUCTION • NETWORK OFF



2. IMPLEMENTATION AGENT

REPOSITORY-ONLY ACCESS

- Edit approved repository files
- Run repository tests
- Create a reviewable diff
- Report validation evidence

NO HOST-WIDE ACCESS • NO PRODUCTION



3. PRODUCTION

HUMAN-CONTROLLED

- No unattended agent access
- Explicit human authorization
- Controlled deployment pipeline
- Monitoring + rollback required

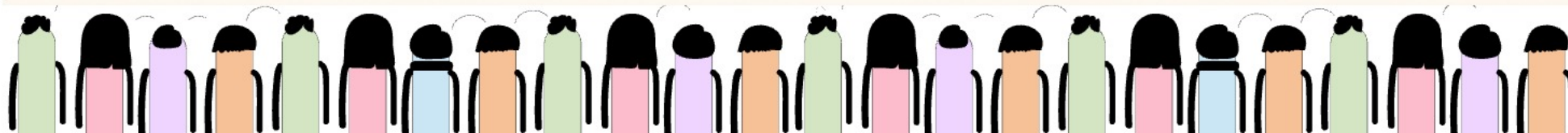
AGENTS DO NOT DEPLOY DIRECTLY



SMALLEST PERMISSION BOUNDARY THAT COMPLETES THE TASK

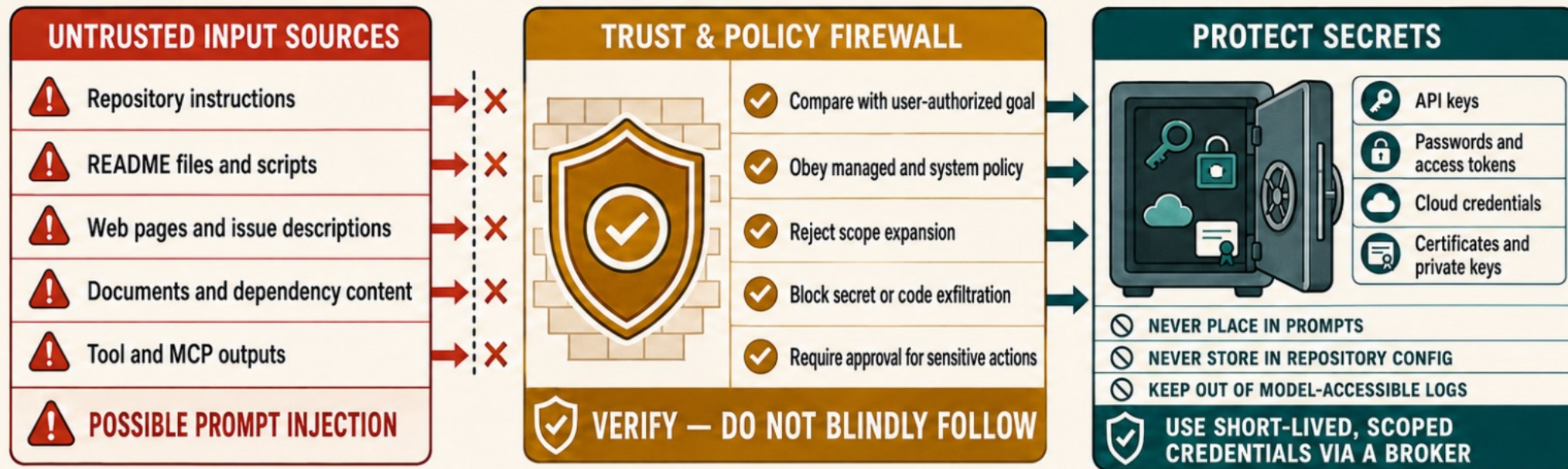


PAYMENT BUG: investigate logs → approve fix → patch repository → tests pass → human-controlled deployment



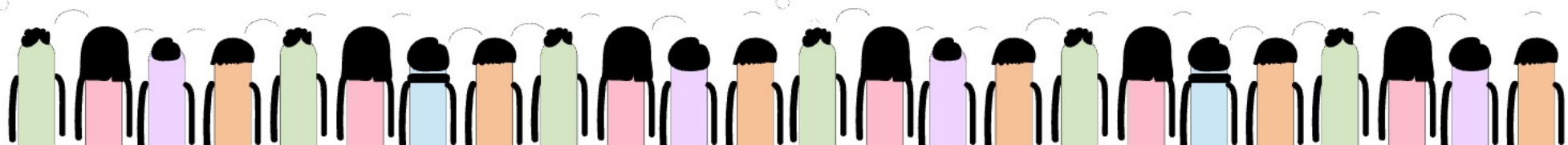
UNTRUSTED INPUTS & SECRET PROTECTION

CONTENT MAY INFORM THE TASK — IT CANNOT EXPAND AUTHORITY



DEFENSE IN DEPTH = UNTRUSTED INPUTS + LEAST PRIVILEGE + SECRET ISOLATION + NETWORK CONTROLS + HUMAN OVERSIGHT

Repository guidance never overrides user authorization, sandbox boundaries, or enterprise requirements.



HANDS-ON LAB 12 — SECURE OPERATING PROFILES

ONE ROLE • ONE PURPOSE • ONE MINIMUM ACCESS BOUNDARY

 **rx-analyst**

 Purpose: Architecture and investigation

 **READ-ONLY**

 Map components and dependencies

 Trace incidents and produce findings

 `default_permissions = ":read-only"`

 **NO FILE CHANGES**

 **rx-developer**

 Purpose: Code modification and testing

 **WORKSPACE WRITE**

 Edit approved repository files

 Run builds and tests

 `default_permissions = ":workspace"`

 **NETWORK OFF • NO PRODUCTION**

 **rx-security**

 Purpose: Adversarial code review

 **READ-ONLY**

 Find attack paths and trust failures

 Report evidence without patching

 `default_permissions = ":read-only"`

 **INDEPENDENT REVIEW**

 **rx-ci**

 Purpose: Non-interactive PR analysis

 **READ-ONLY • NO APPROVALS**

 Run codex exec in CI

 Return machine-readable findings

 `default_permissions = ":read-only"`
`approval_policy = "never"`

 **FAIL CLOSED • NO WRITES**

 **rx-remediator** 

 Purpose: Controlled patch creation

 **WORKSPACE WRITE • ISOLATED RUNNER**

 Create a bounded security patch

 Run targeted validation

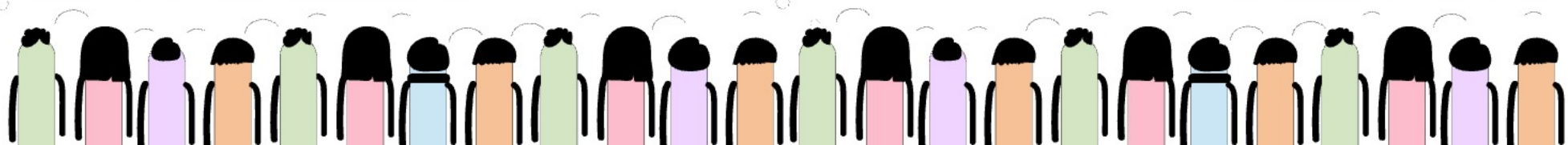
 `default_permissions = ":workspace"`

 **HUMAN REVIEW BEFORE MERGE**

 `~/codex/<profile>.config.toml` →  `codex --profile <profile>` →  **TASK-SPECIFIC SESSION**

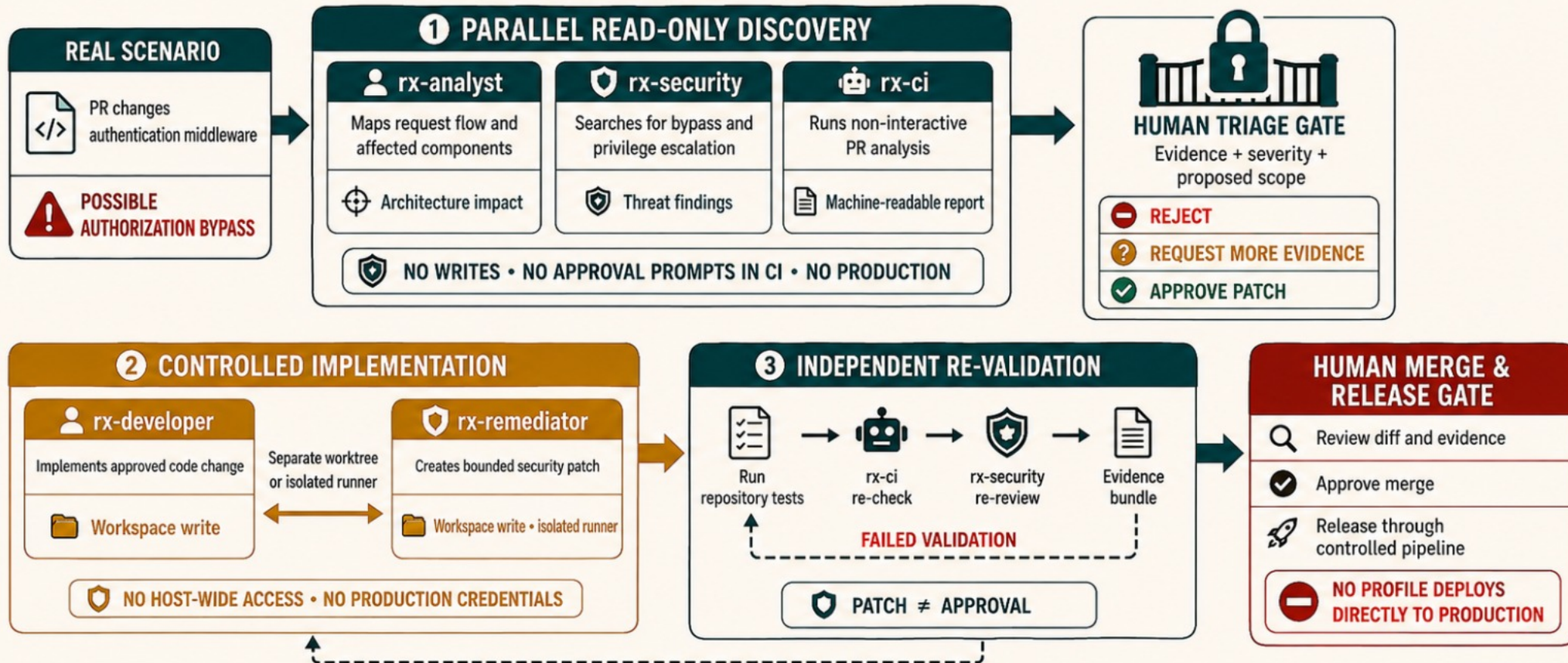
 **ALL PROFILES: NO DANGER-FULL-ACCESS • NO PRODUCTION ACCESS • NO MODEL-VISIBLE SECRETS**

Permission profiles are beta; verify the supported Codex client version.



SECURE MULTI-PROFILE PR WORKFLOW

READ-ONLY DISCOVERY → CONTROLLED CHANGE → INDEPENDENT VALIDATION



SEPARATION OF DUTIES = INVESTIGATE • IMPLEMENT • VERIFY • AUTHORIZE

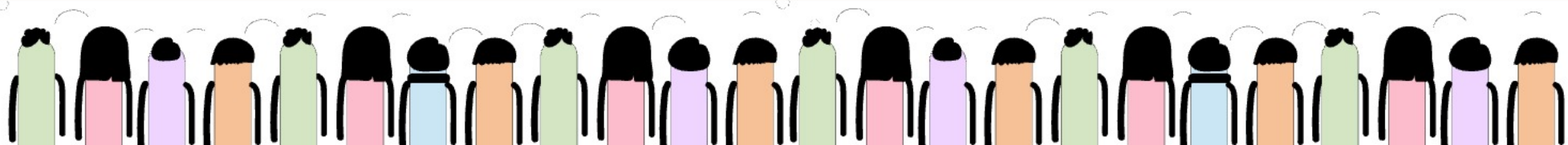
`> codex --profile rx-analyst`

`> codex --profile rx-security`

`> codex exec --profile rx-ci`

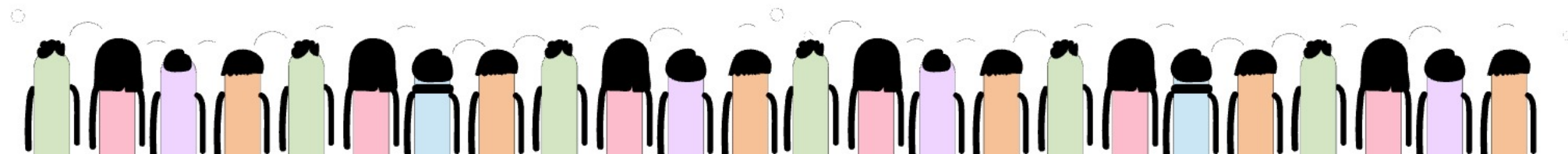
`> codex --profile rx-developer`

`> codex --profile rx-remediator`



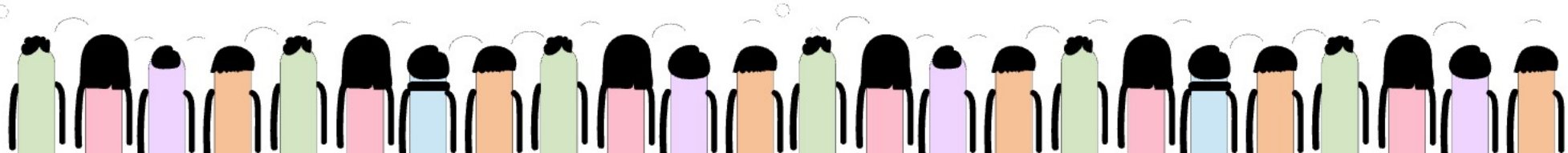
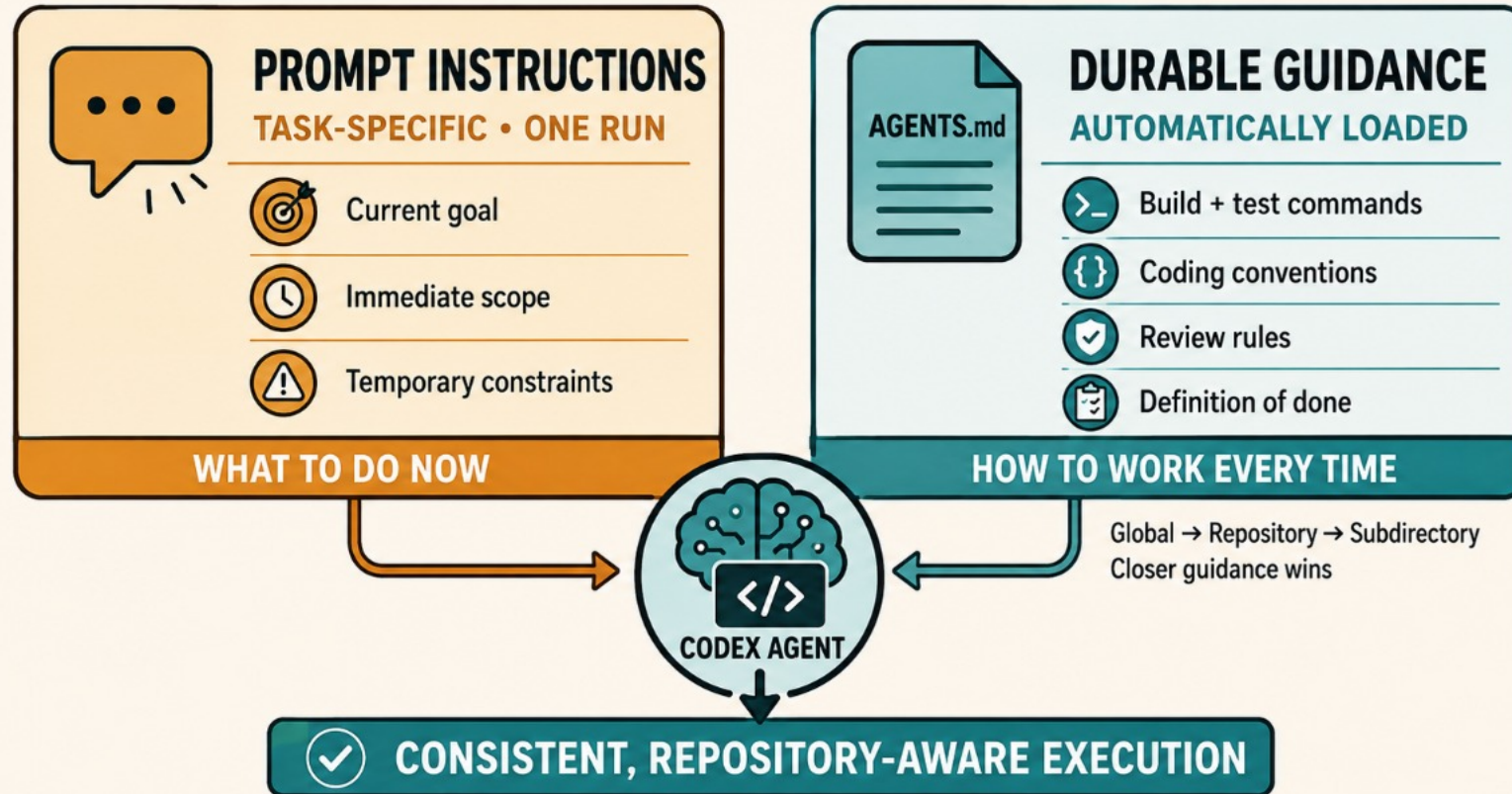


Codex



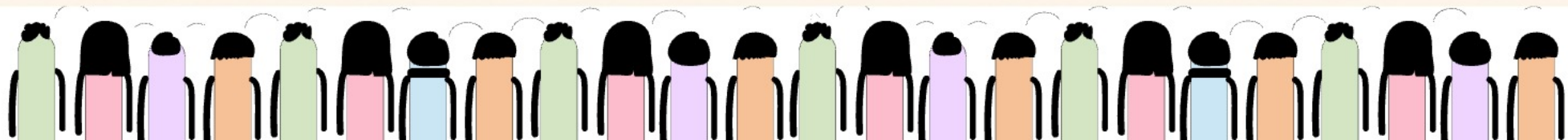
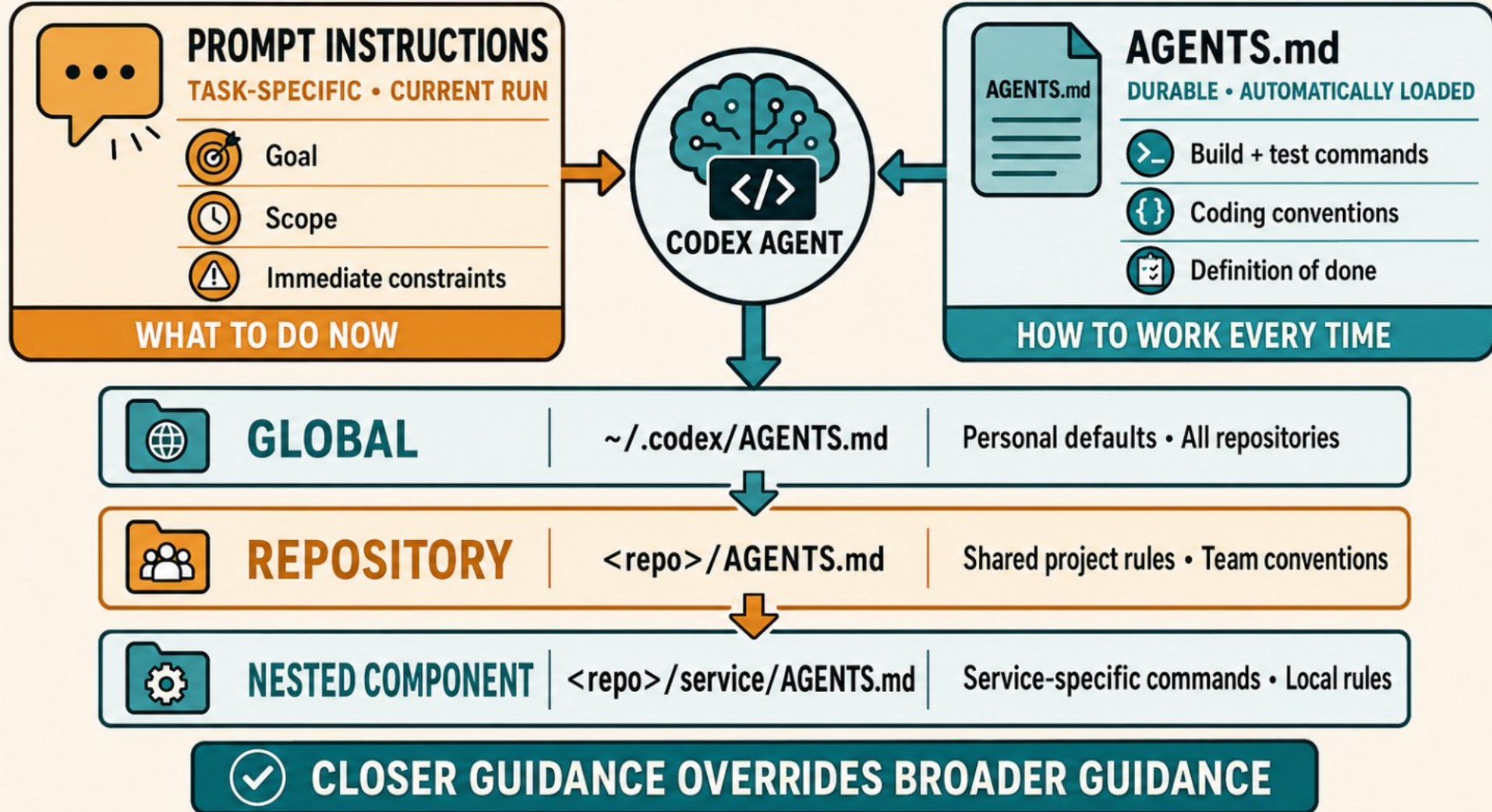
Prompt

PROMPT INSTRUCTIONS vs AGENTS.md



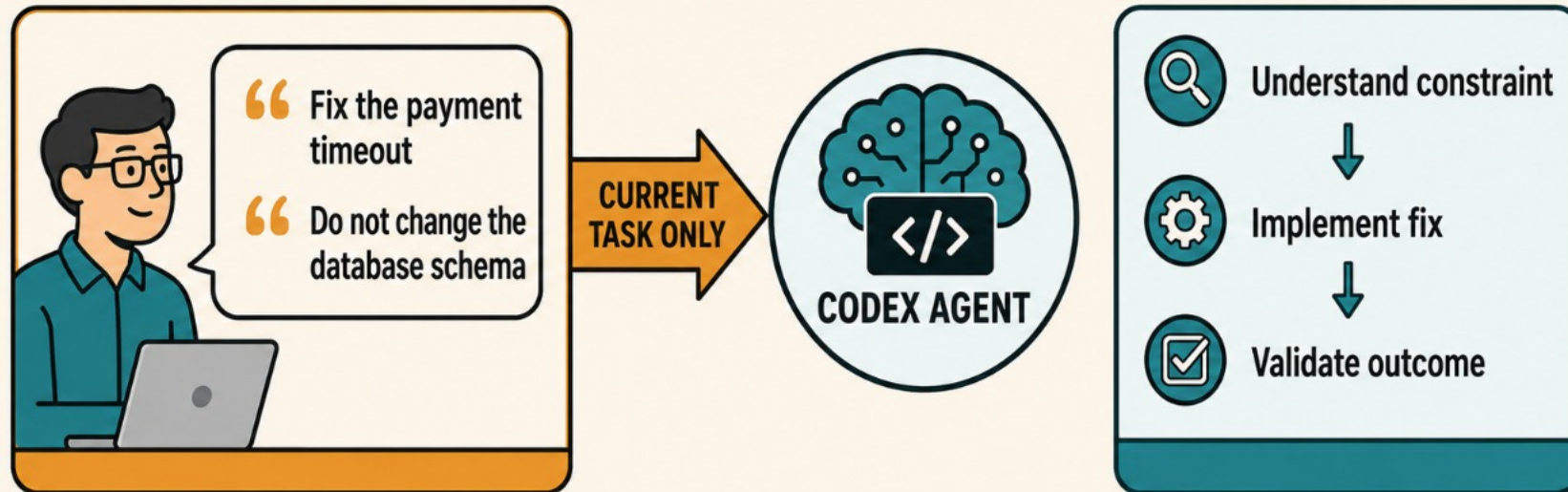
Prompt

CODEX INSTRUCTION SYSTEM

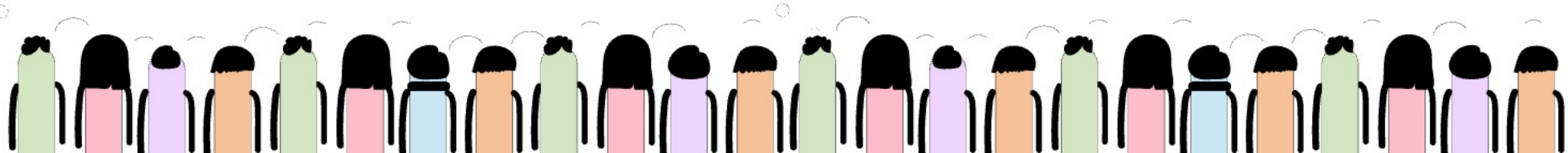


Prompt

1. PROMPT — ONE-TIME CONSTRAINT

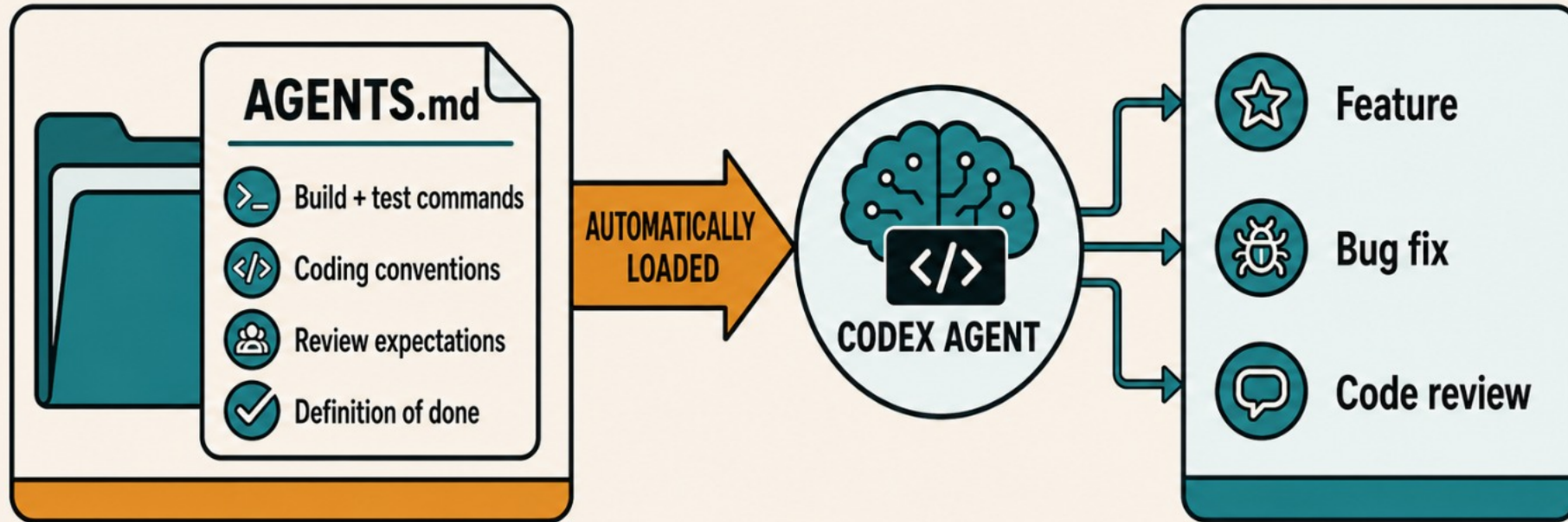


USE A PROMPT FOR A ONE-TIME GOAL, SCOPE OR CONSTRAINT

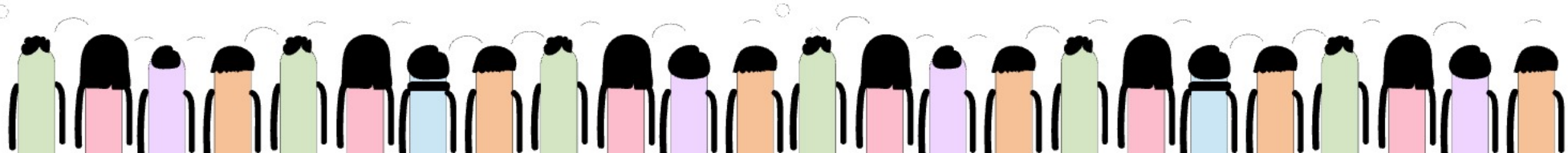


AGENTS

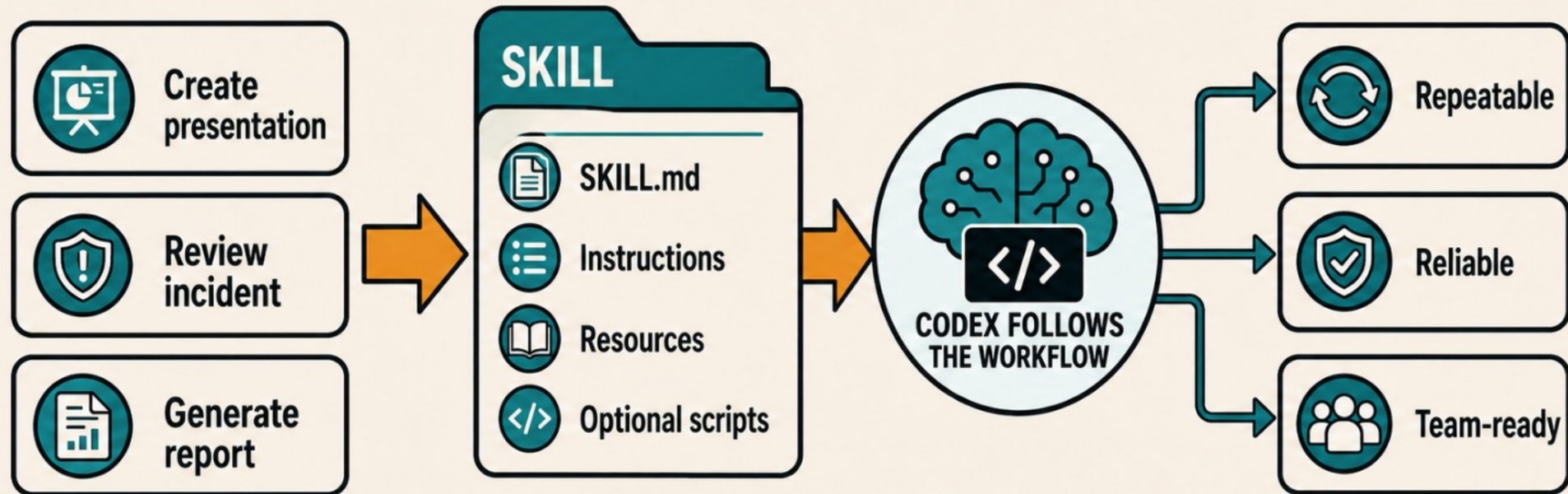
2. AGENTS.md – REPOSITORY STANDARDS



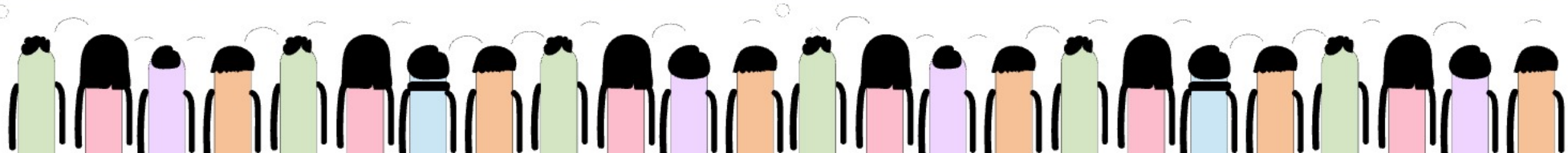
USE AGENTS.md FOR DURABLE RULES CODEX SHOULD FOLLOW IN THIS REPOSITORY



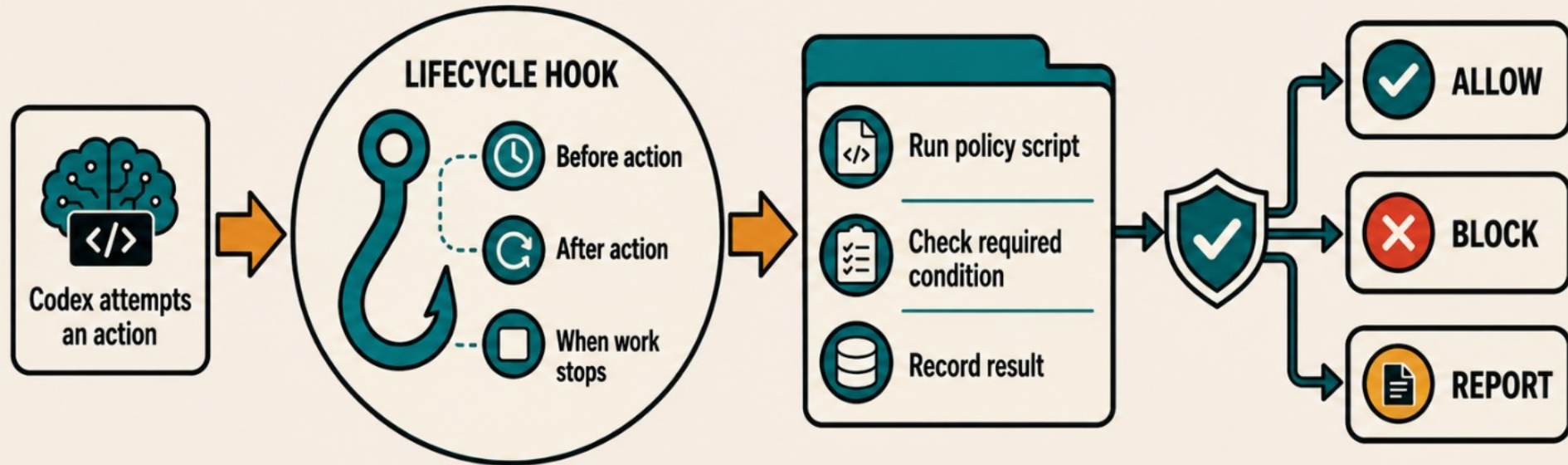
3. SKILL — REUSABLE WORKFLOW



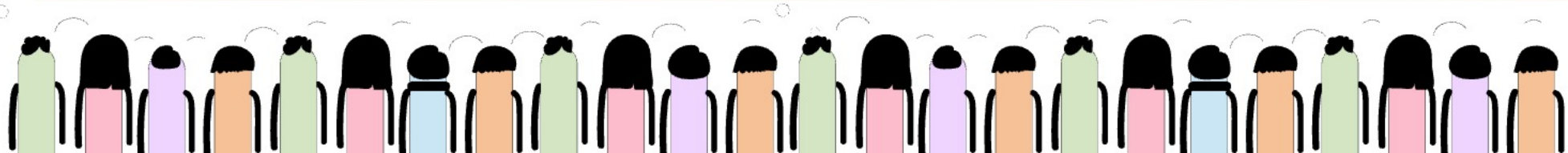
USE A SKILL WHEN THE SAME MULTI-STEP WORKFLOW SHOULD BE REUSED



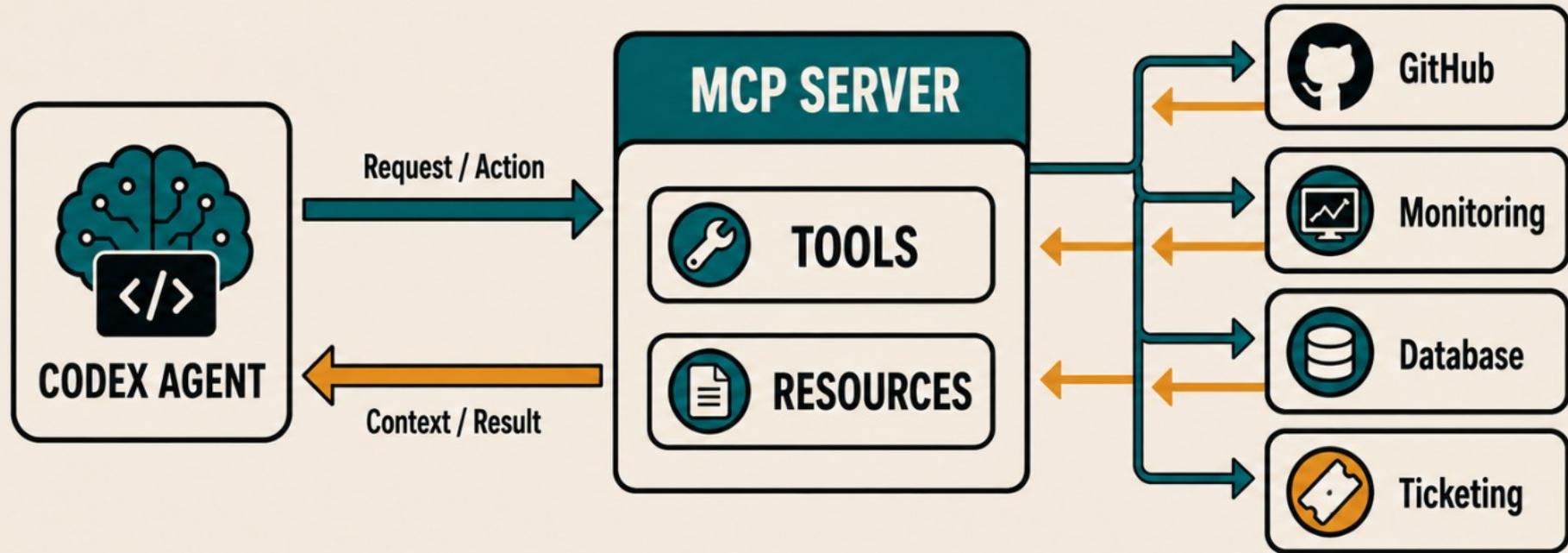
4. HOOK — LIFECYCLE ENFORCEMENT



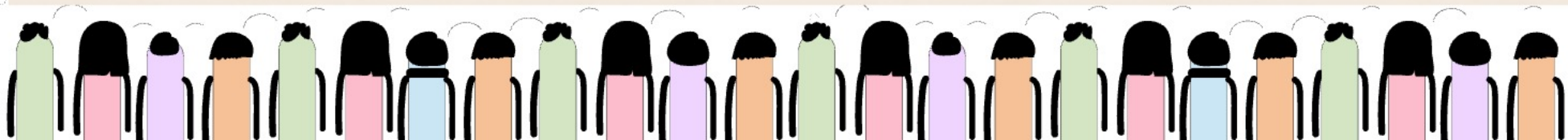
USE A HOOK TO RUN DETERMINISTIC CHECKS AT CODEX LIFECYCLE EVENTS



5. MCP — EXTERNAL INFORMATION OR ACTION

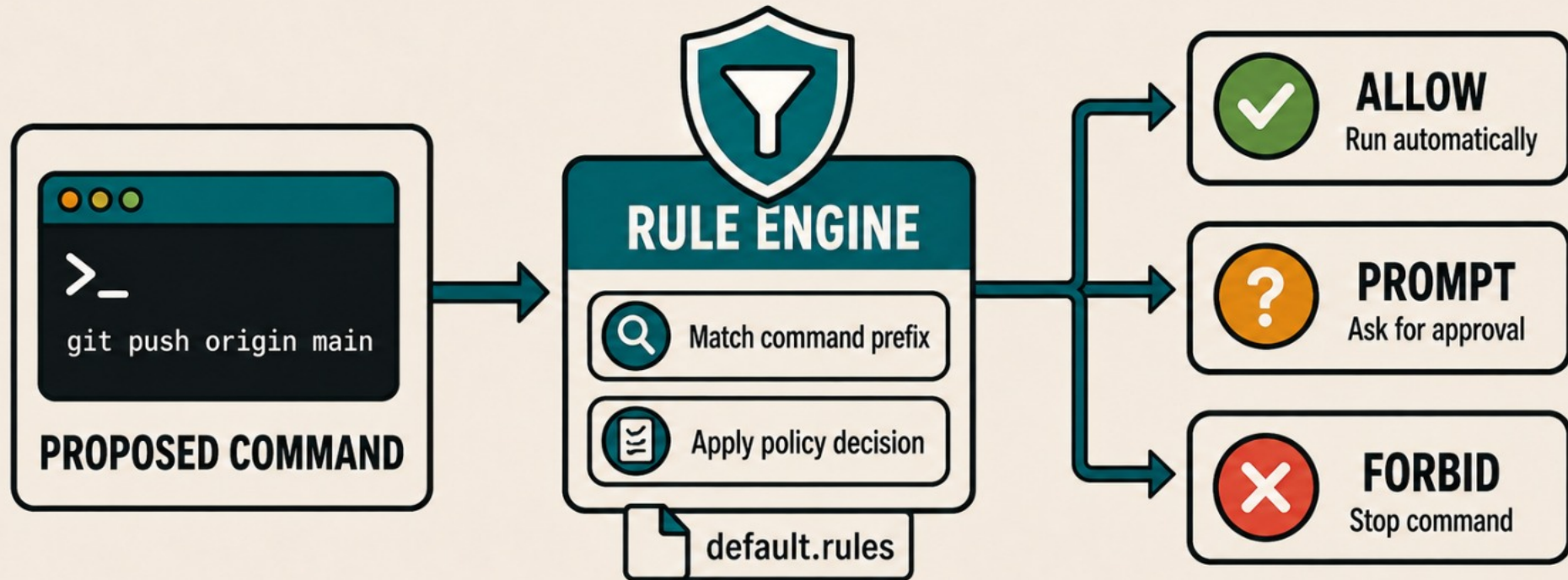


USE MCP TO CONNECT CODEX WITH EXTERNAL TOOLS, DATA AND SYSTEMS

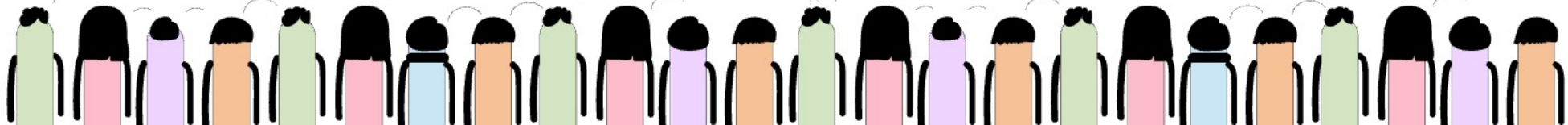


Rule

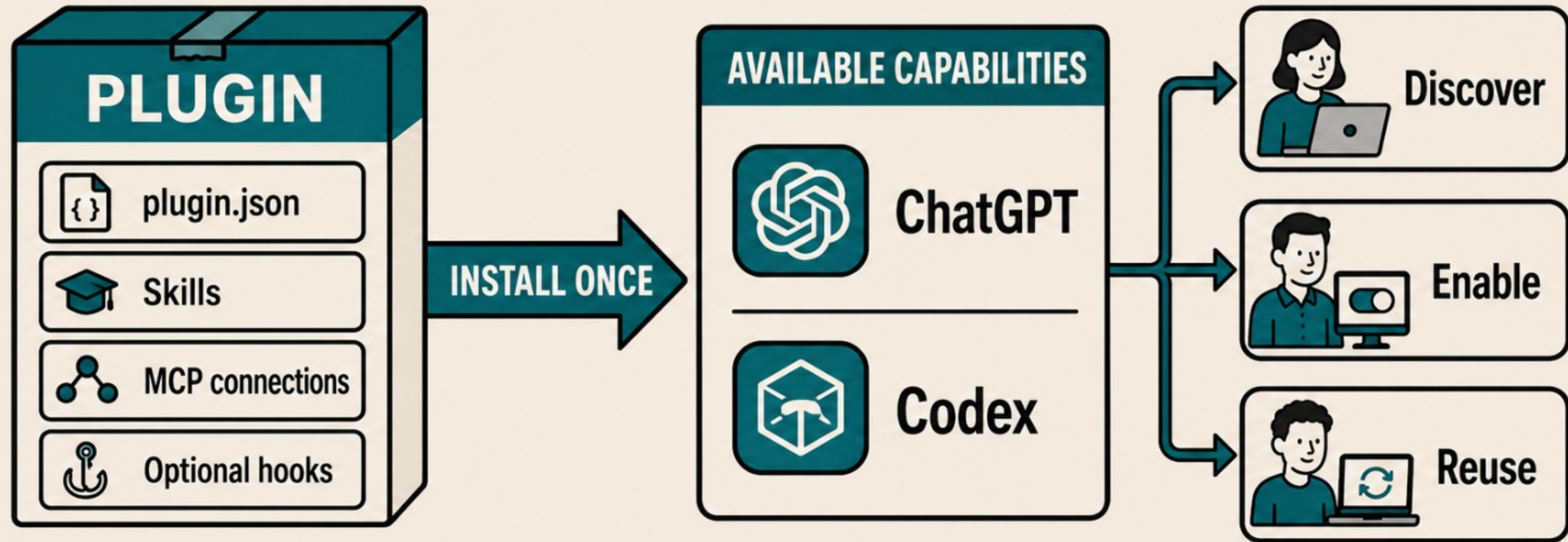
6. RULE — COMMAND POLICY



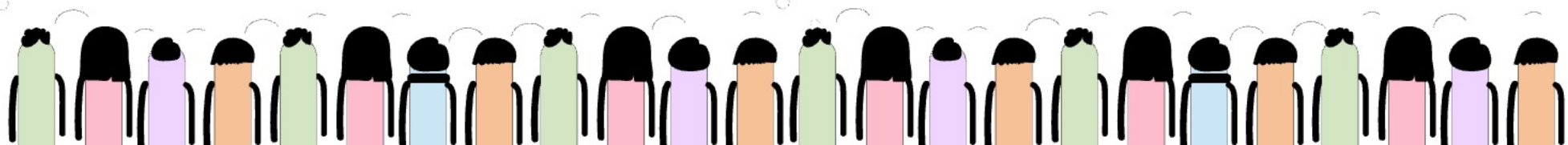
USE A RULE TO CONTROL WHICH COMMANDS CODEX MAY RUN



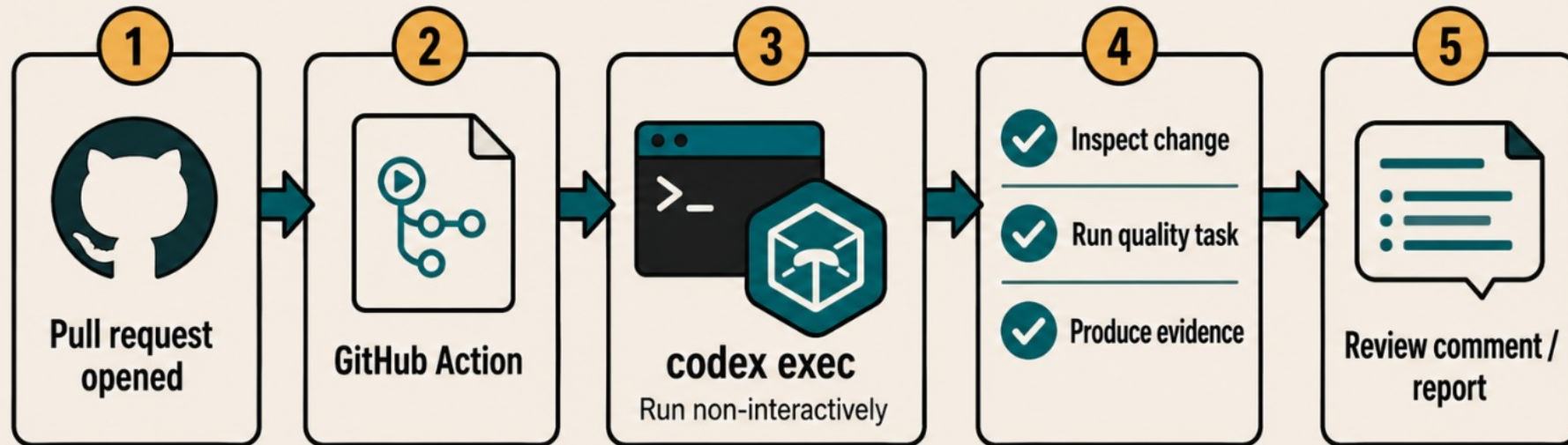
7. PLUGIN — INSTALLABLE CAPABILITY BUNDLE



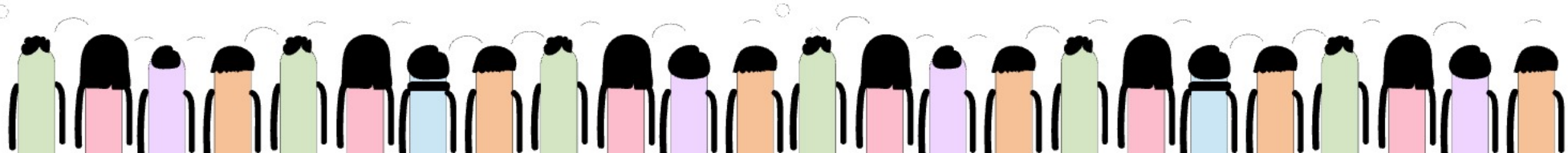
USE A PLUGIN TO DISTRIBUTE AN INSTALLABLE BUNDLE OF SKILLS AND CONNECTIONS



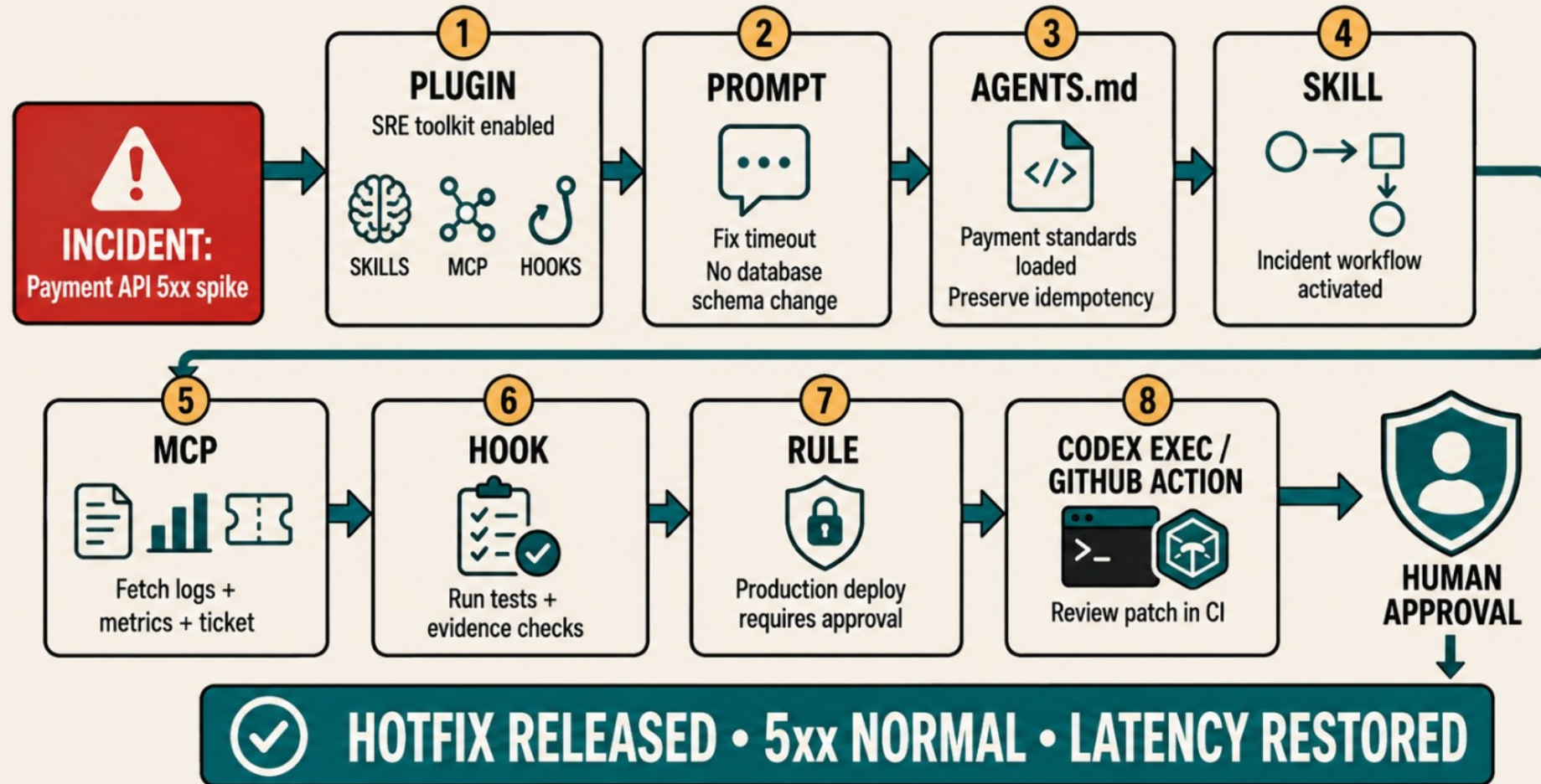
8. CODEX EXEC / GITHUB ACTION — AUTOMATED PIPELINE TASK



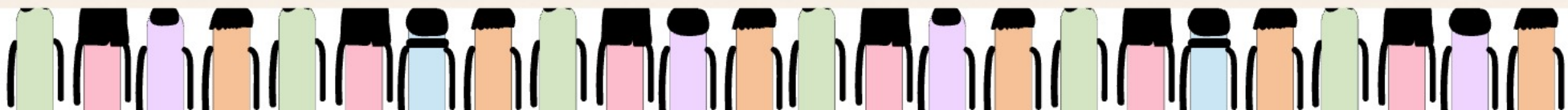
**USE CODEX EXEC OR THE GITHUB ACTION FOR REPEATABLE
AUTOMATED ENGINEERING TASKS**



9. REAL-TIME SCENARIO — PAYMENT API PRODUCTION HOTFIX

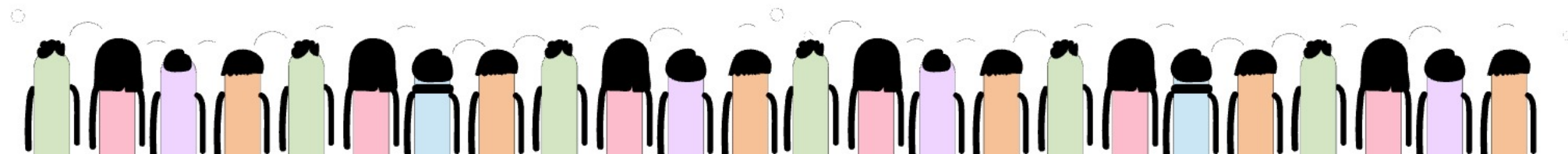


Codex investigates, changes, validates and reports — humans retain production authority





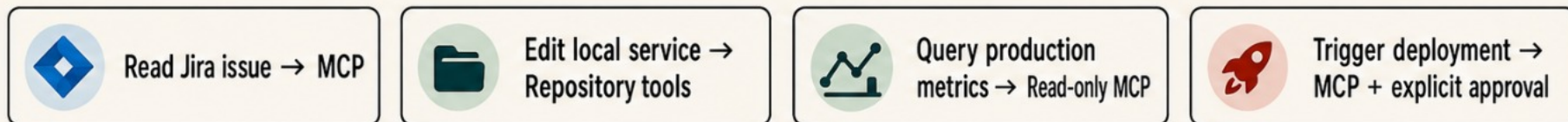
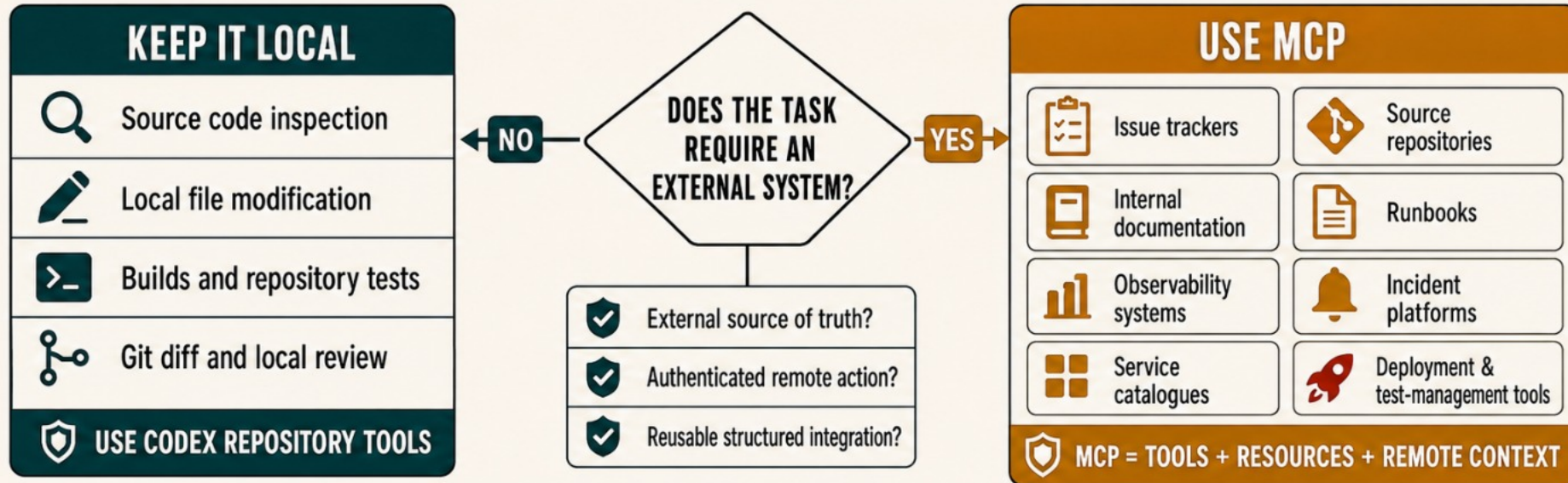
Codex



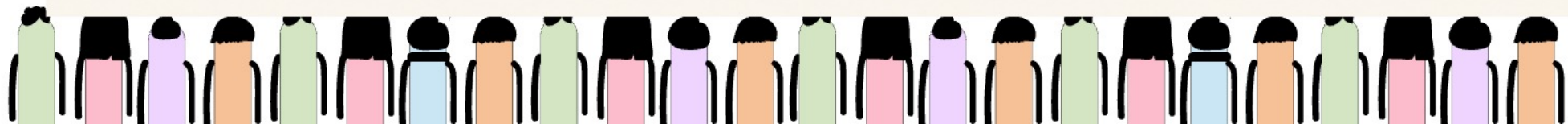
MCP -Module17

MODULE 17 — WHEN MCP IS THE RIGHT ANSWER

USE MCP WHEN THE NEEDED CONTEXT OR ACTION LIVES OUTSIDE THE LOCAL REPOSITORY



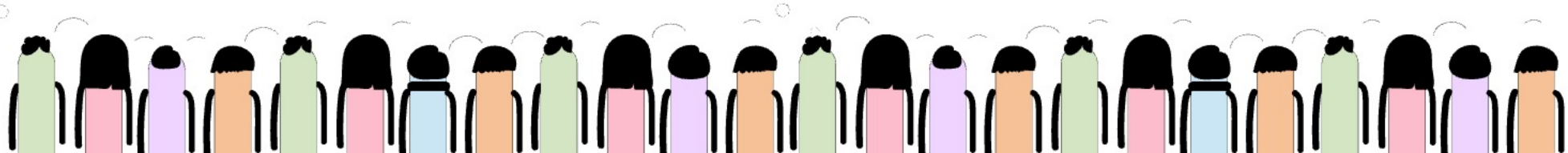
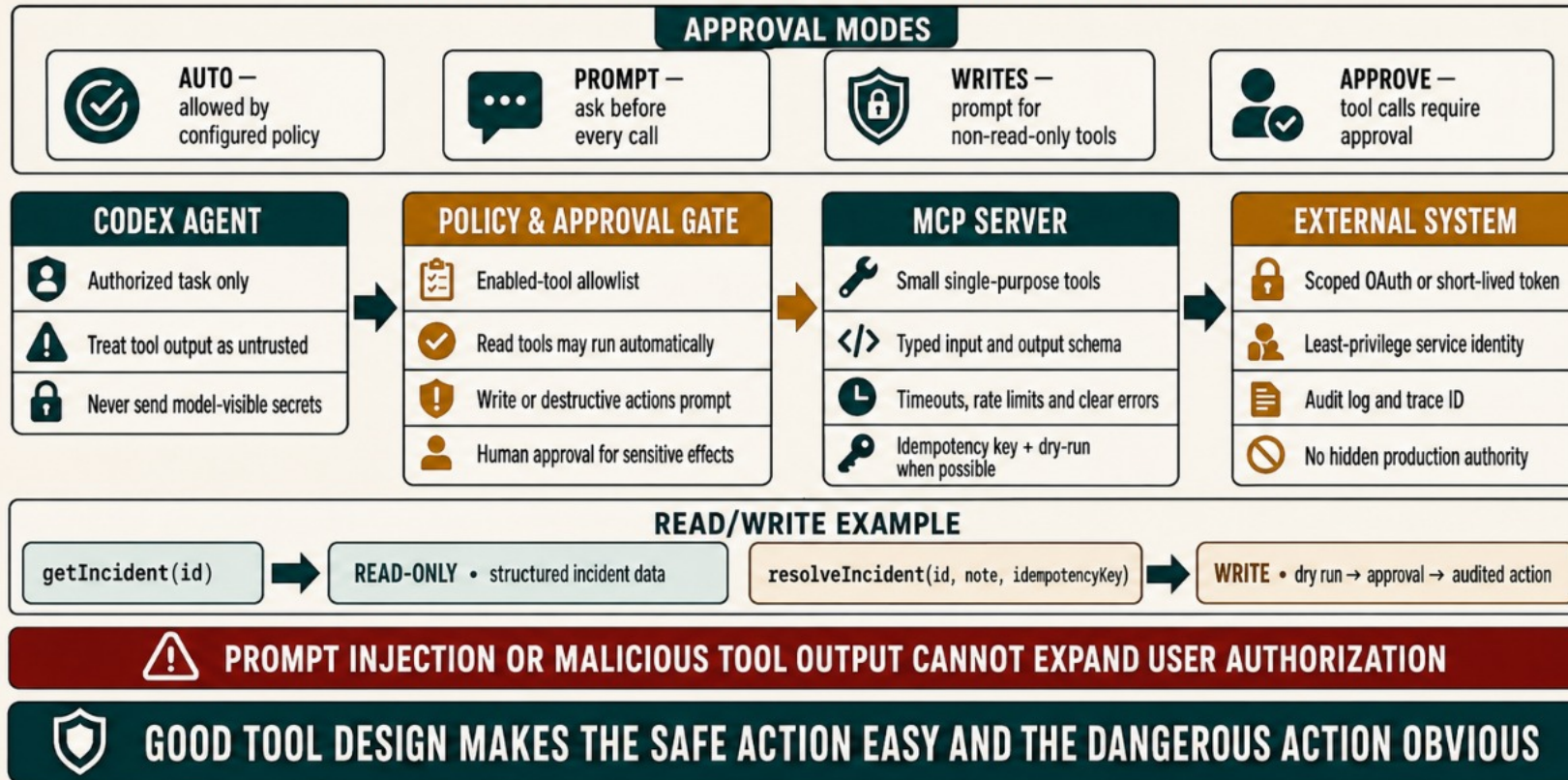
MCP CONNECTS CODEX TO EXTERNAL CAPABILITIES — IT DOES NOT REPLACE A CLEAR WORKFLOW



MCP -Module17

SECURE MCP TOOL DESIGN FOR CODEX

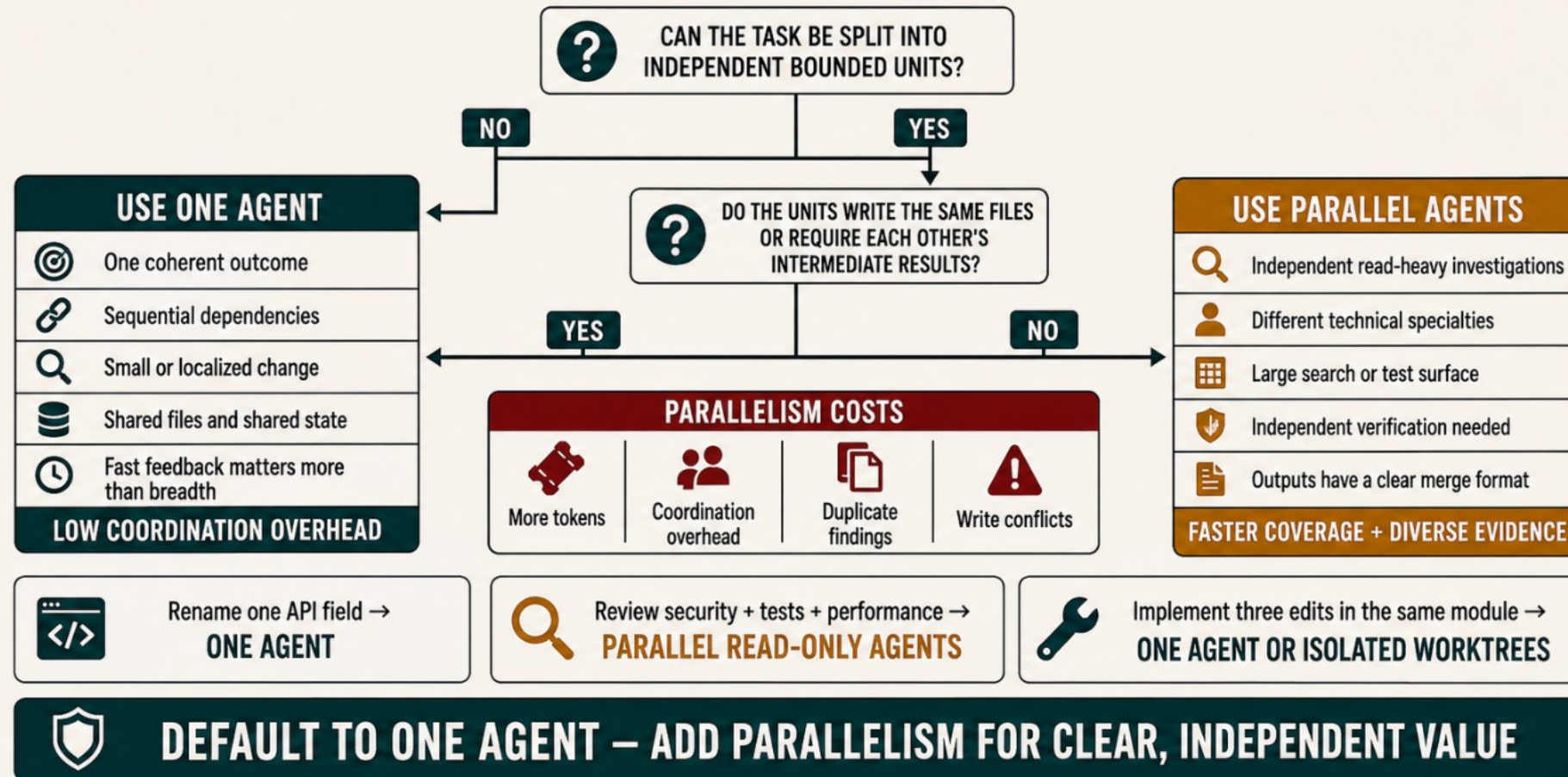
CAPABILITY MUST BE NARROW, EXPLICIT, REVIEWABLE AND REVOCABLE



MCP -Module17

ONE AGENT OR PARALLEL AGENTS?

PARALLELISM CREATES VALUE ONLY WHEN TASKS ARE INDEPENDENT AND MERGEABLE



MCP -Module17

BOUNDED TASK DECOMPOSITION

READ-ONLY SPECIALISTS RETURN EVIDENCE — THE COORDINATOR OW'S SYNTHESIS

INCIDENT: Checkout latency increased after retry-policy change



COORDINATING AGENT

- Define non-overlapping scopes
- Set access = read-only
- Specify evidence and return schema
- Wait for all specialists



ARCHITECTURE SPECIALIST



Scope: Trace checkout → payment dependencies



Return: Call path + risk boundaries

READ-ONLY • NO PATCHES



LOG SPECIALIST



Scope: Analyze timeout and retry patterns



Return: Timeline + correlated evidence

READ-ONLY • NO PATCHES



TEST SPECIALIST



Scope: Find missing latency and failure tests



Return: Test gaps + reproduction steps

READ-ONLY • NO PATCHES



SECURITY SPECIALIST



Scope: Check retry abuse and trust boundaries



Return: Attack paths + severity

READ-ONLY • NO PATCHES

NORMALIZED RETURN FORMAT



Finding ID



Claim



Evidence



Confidence



Recommended next action



BAD: Fix checkout latency



GOOD: Analyze retry behavior in payment client; read-only; return file references, evidence and confidence

COORDINATOR SYNTHESIS



Compare conclusions



Resolve contradictions



Remove duplicates

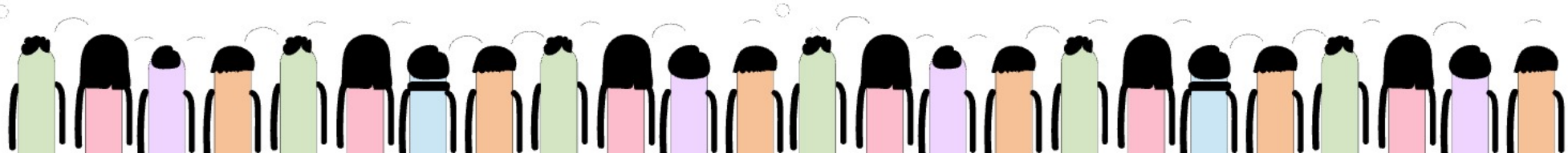


Produce one prioritized report



HUMAN APPROVES IMPLEMENTATION SCOPE

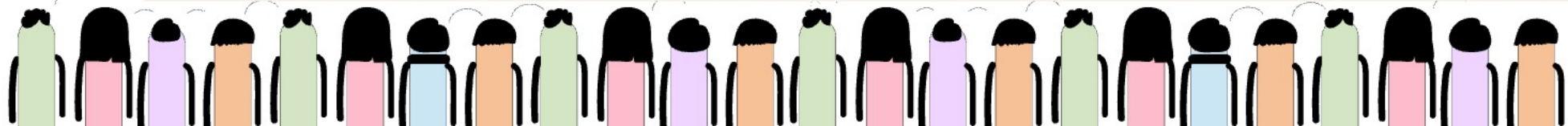
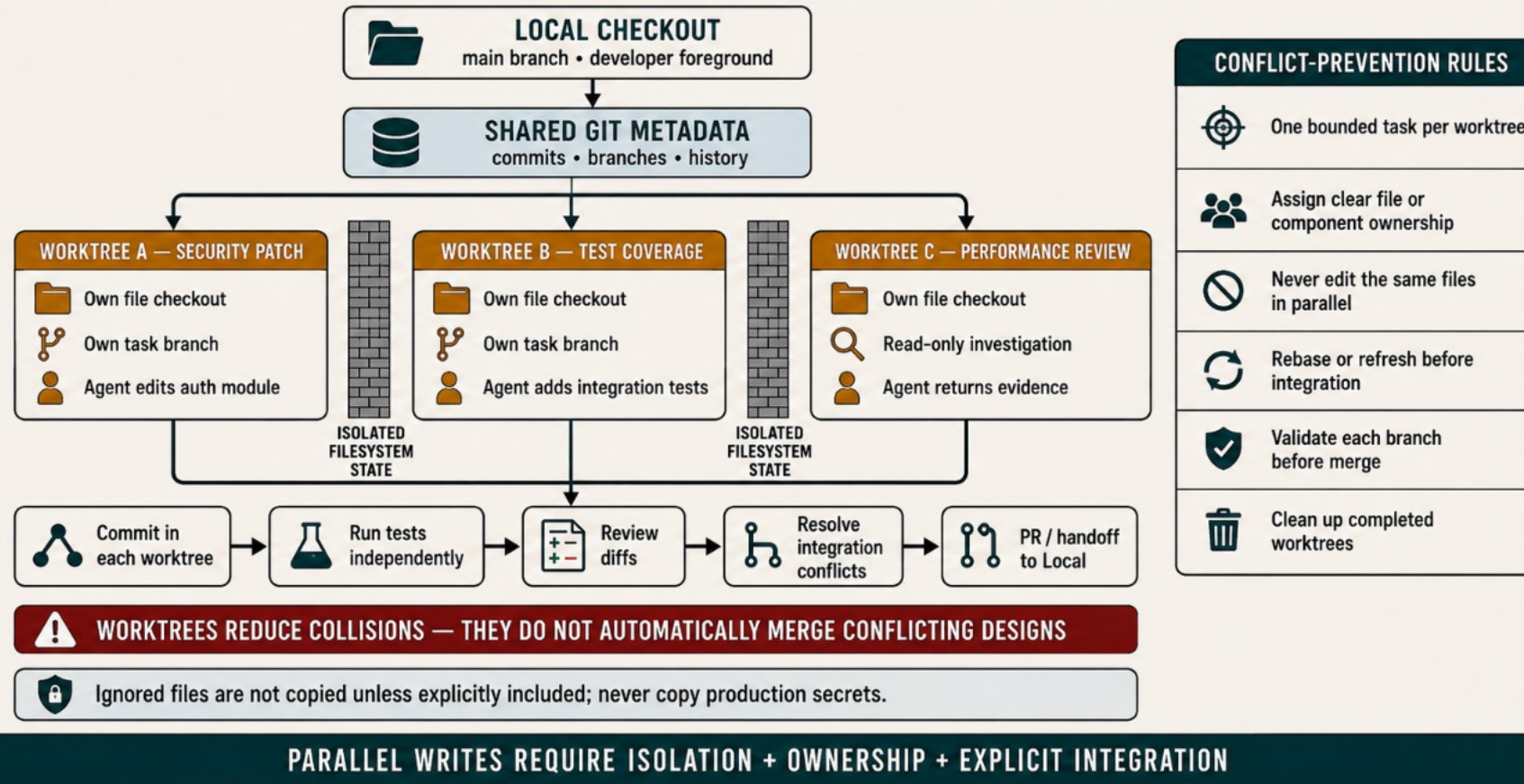
BOUNDARIES REDUCE CONTEXT NOISE, DUPLICATE WORK AND ACCIDENTAL WRITES



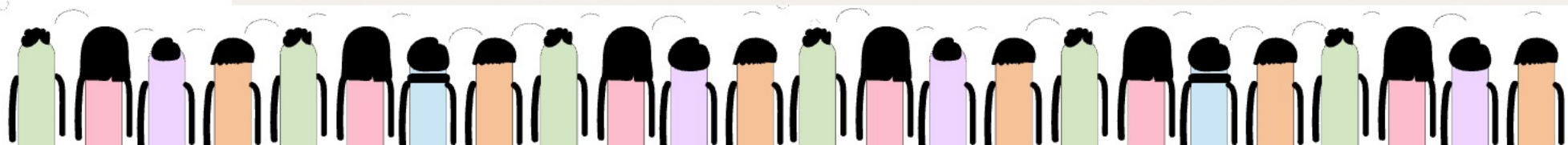
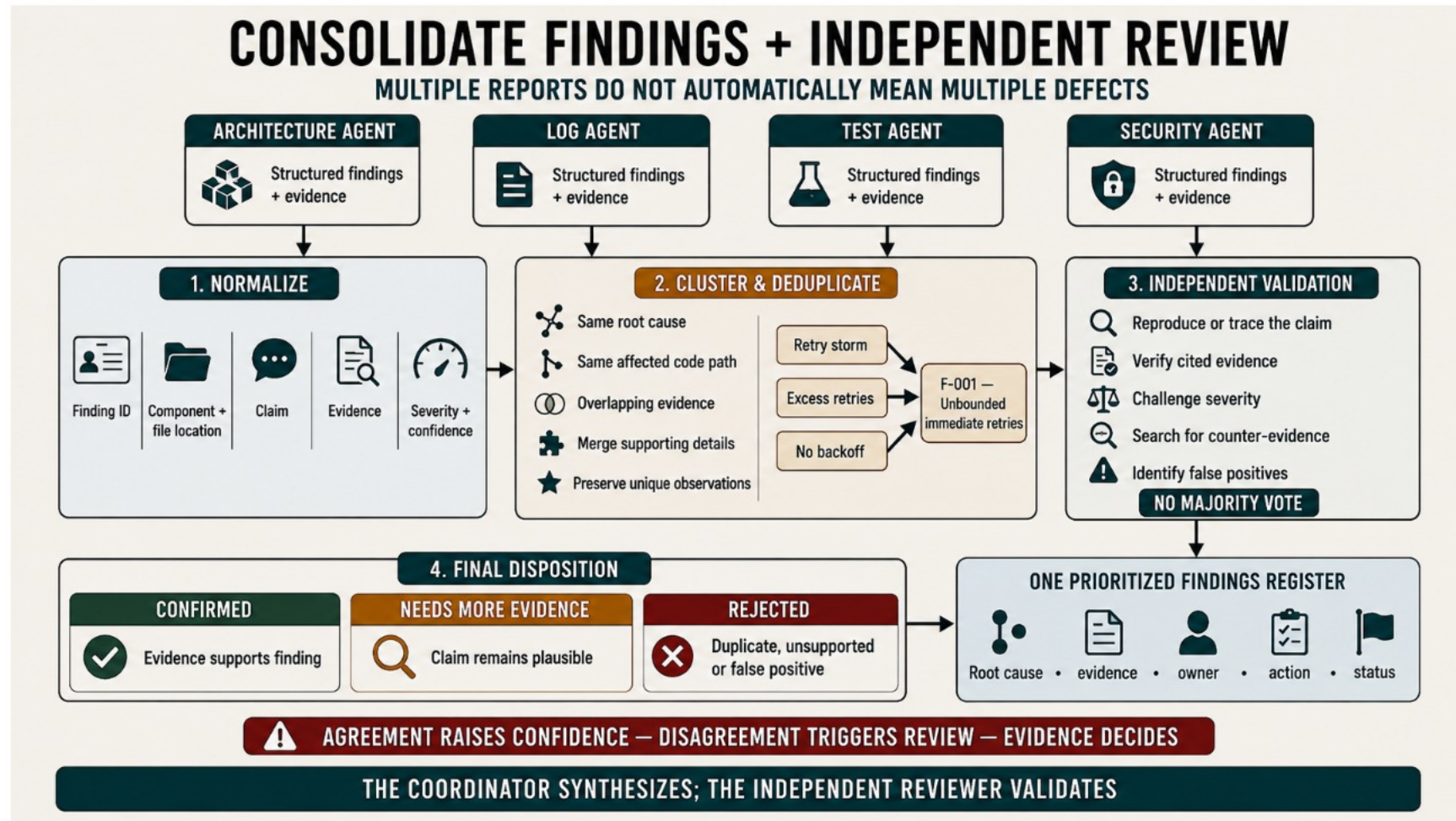
MCP -Module17

GIT WORKTREE ISOLATION FOR PARALLEL AGENTS

SEPARATE CHECKOUTS PREVENT WORKING-DIRECTORY COLLISIONS

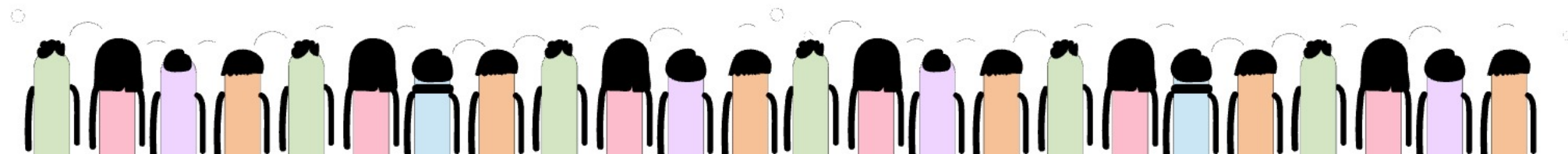


MCP -Module17



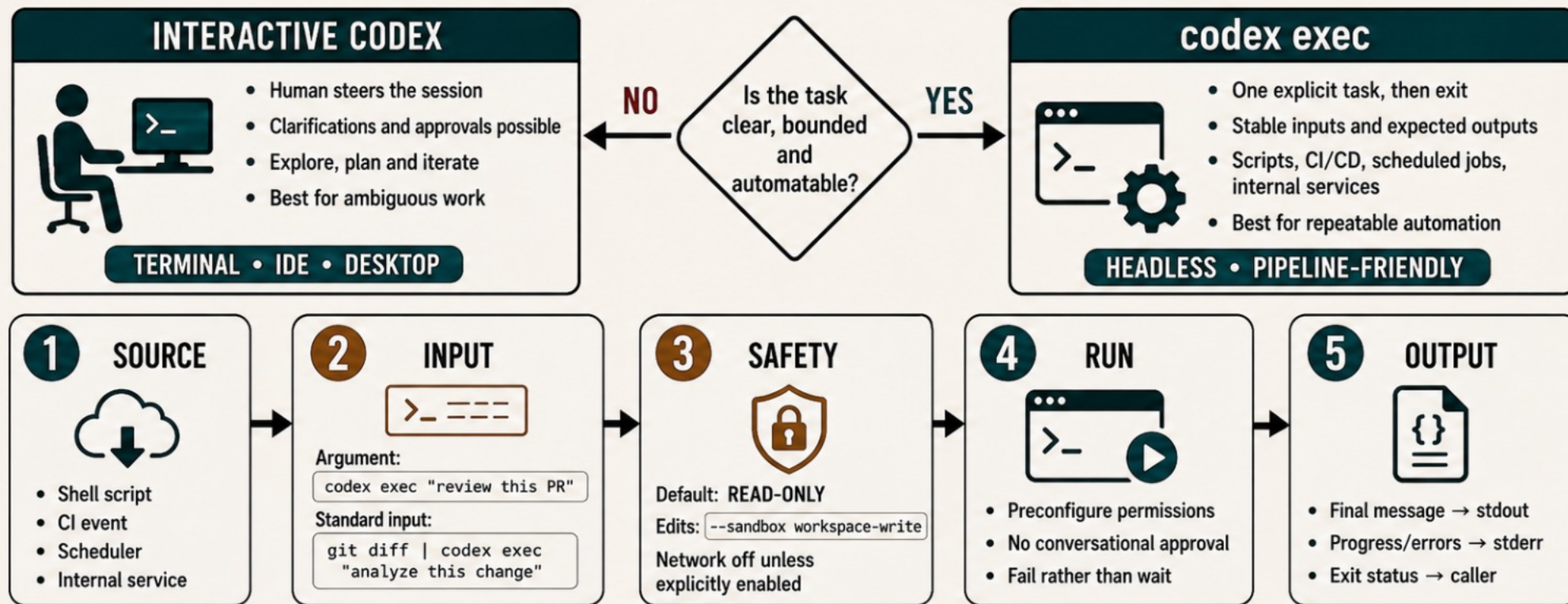


Codex



MODULE 19 – WHEN TO USE codex exec

NON-INTERACTIVE, REPEATABLE EXECUTION FOR AUTOMATION

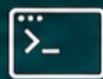


UNATTENDED RULE — DESIGN THE JOB SO IT NEVER NEEDS AN INTERACTIVE APPROVAL



REAL SCENARIO:

Nightly dependency review



read-only
codex exec



JSON report

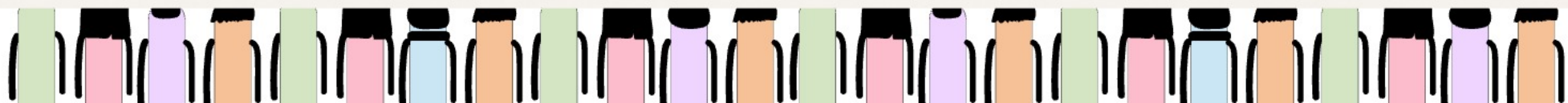


CI artifact



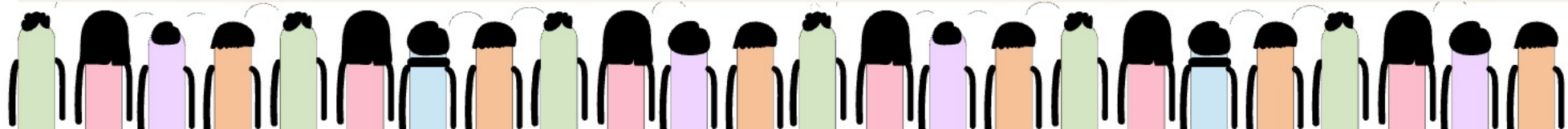
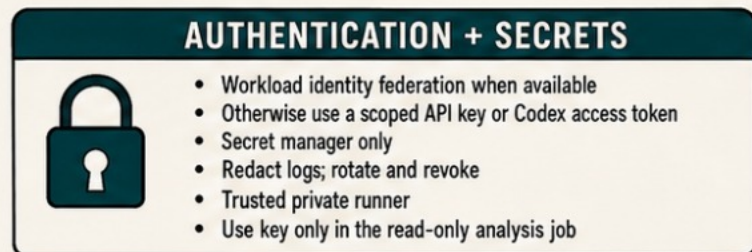
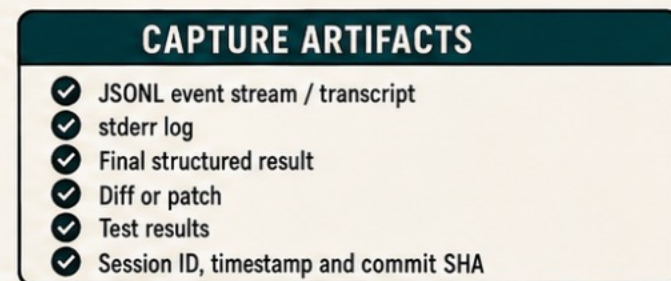
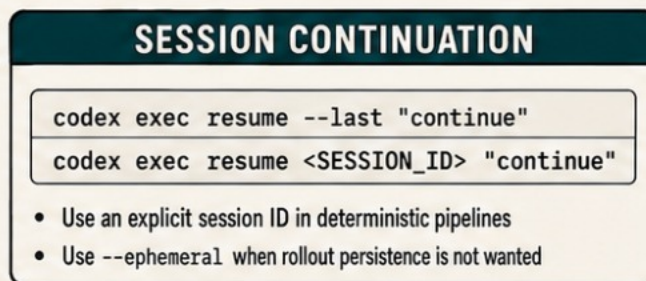
human triage

AUTOMATION = BOUNDED TASK + EXPLICIT SANDBOX + DETERMINISTIC INPUT + CHECKED OUTPUT



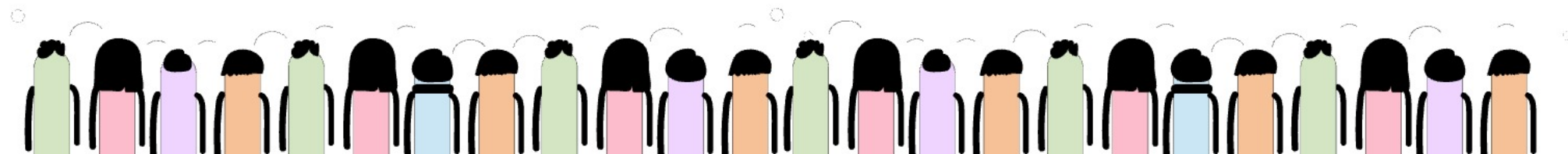
PRODUCTION-GRADE codex exec AUTOMATION

STRUCTURED OUTPUT • RECOVERY • ARTIFACTS • AUTHENTICATION • GIT SAFETY



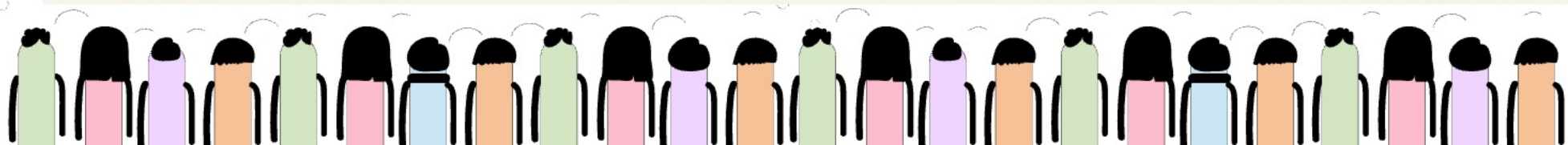
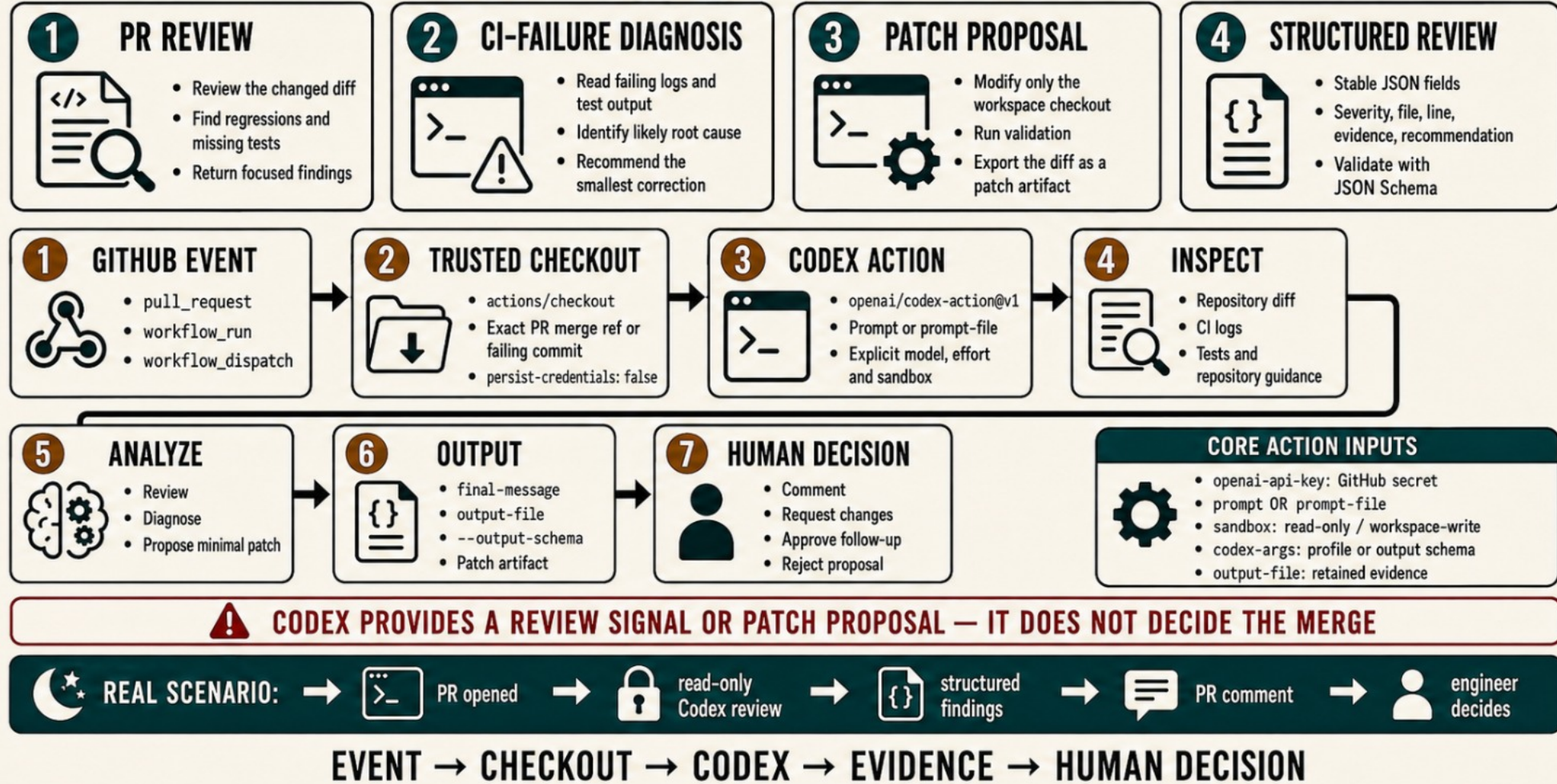


Codex



MODULE 20 — CODEX GITHUB ACTION

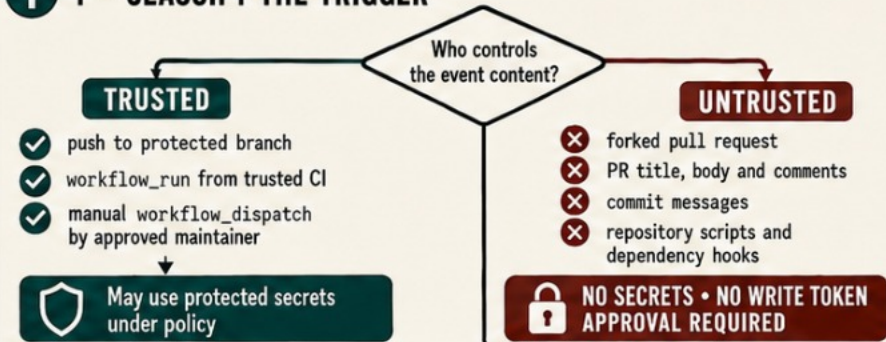
FROM GITHUB EVENT TO REVIEWABLE CI EVIDENCE



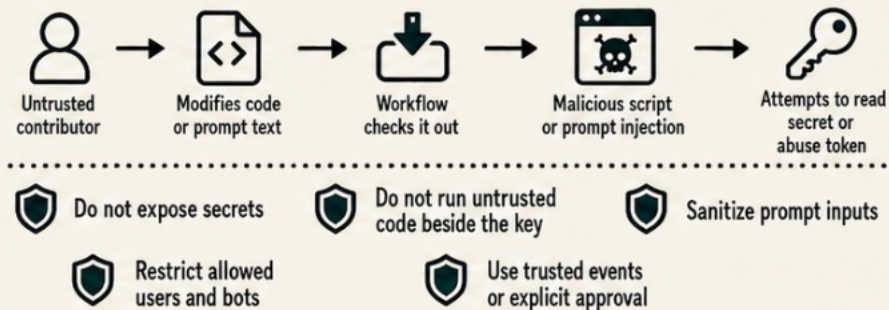
SECURE CODEX CI CONTROL PLANE

TRUST THE EVENT FIRST — THEN GRANT THE MINIMUM AUTHORITY

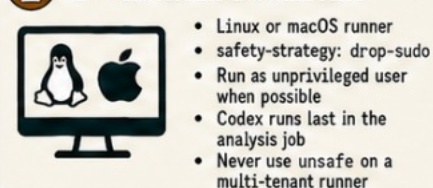
1 — CLASSIFY THE TRIGGER



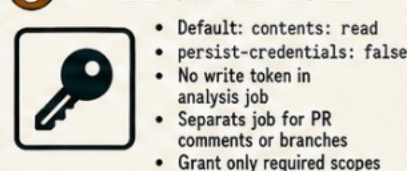
! FORKED PR ATTACK PATH



2 — RUNNER BOUNDARY



3 — REPOSITORY TOKEN



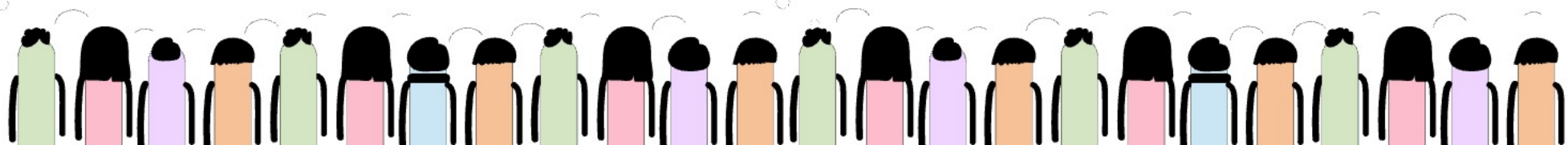
4 — BRANCH PROTECTION



5 — ARTIFACT POLICY

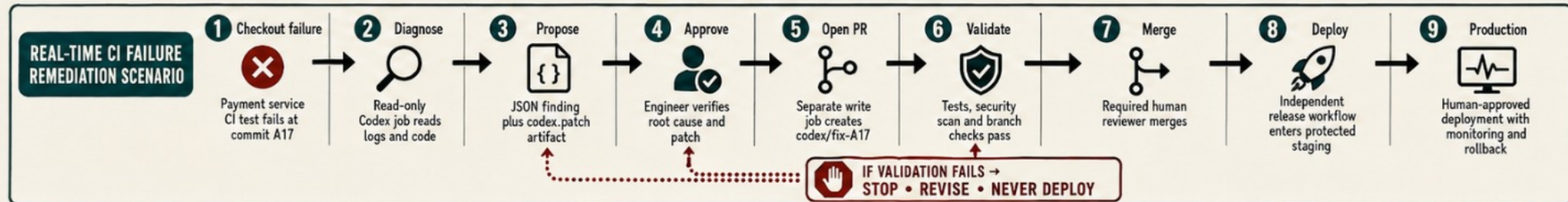
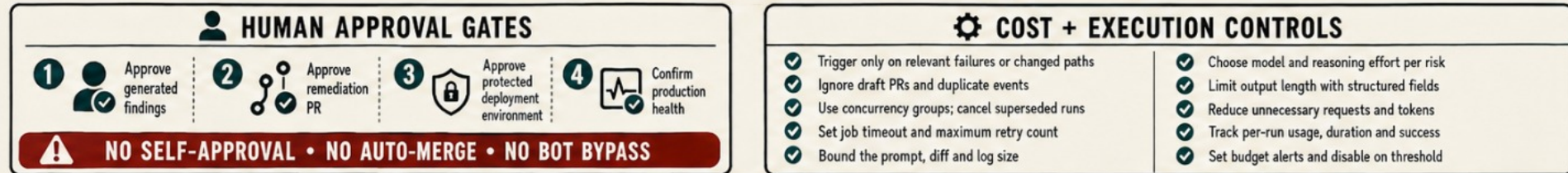
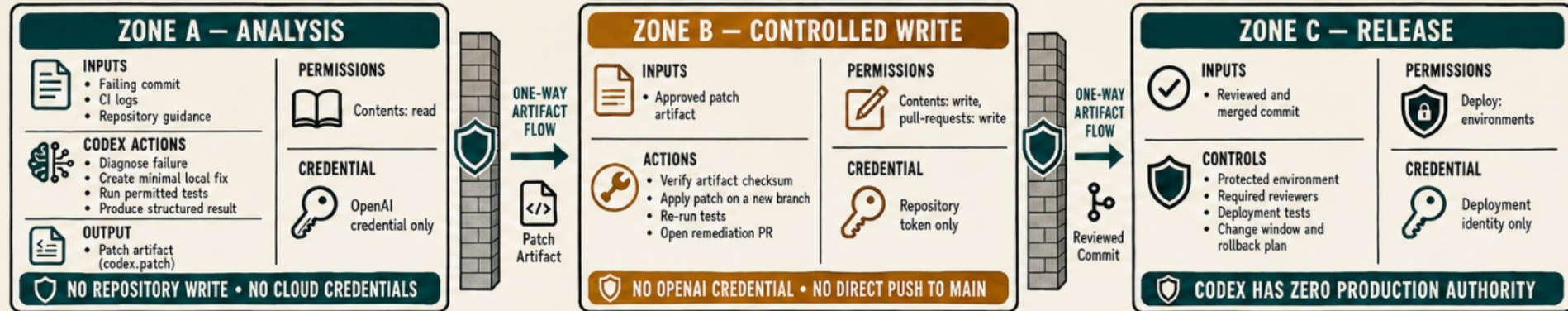


MINIMUM PERMISSION: EVENT TRUST + READ-ONLY ANALYSIS + SEPARATE WRITE JOB + HUMAN GATE



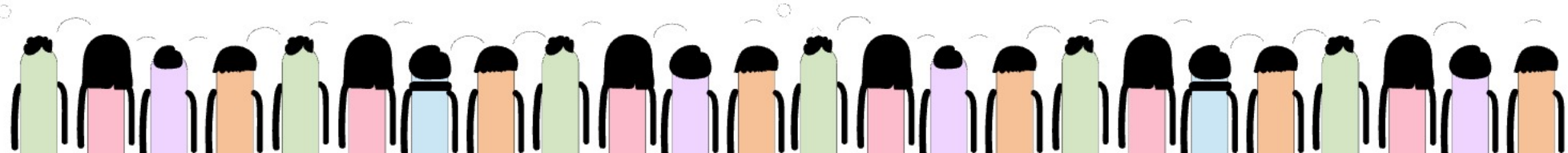
CONTROLLED CI REMEDIATION — NEVER AUTONOMOUS DEPLOYMENT

CODEX MAY ANALYZE AND PROPOSE • PEOPLE AND DELIVERY CONTROLS AUTHORIZE RELEASE



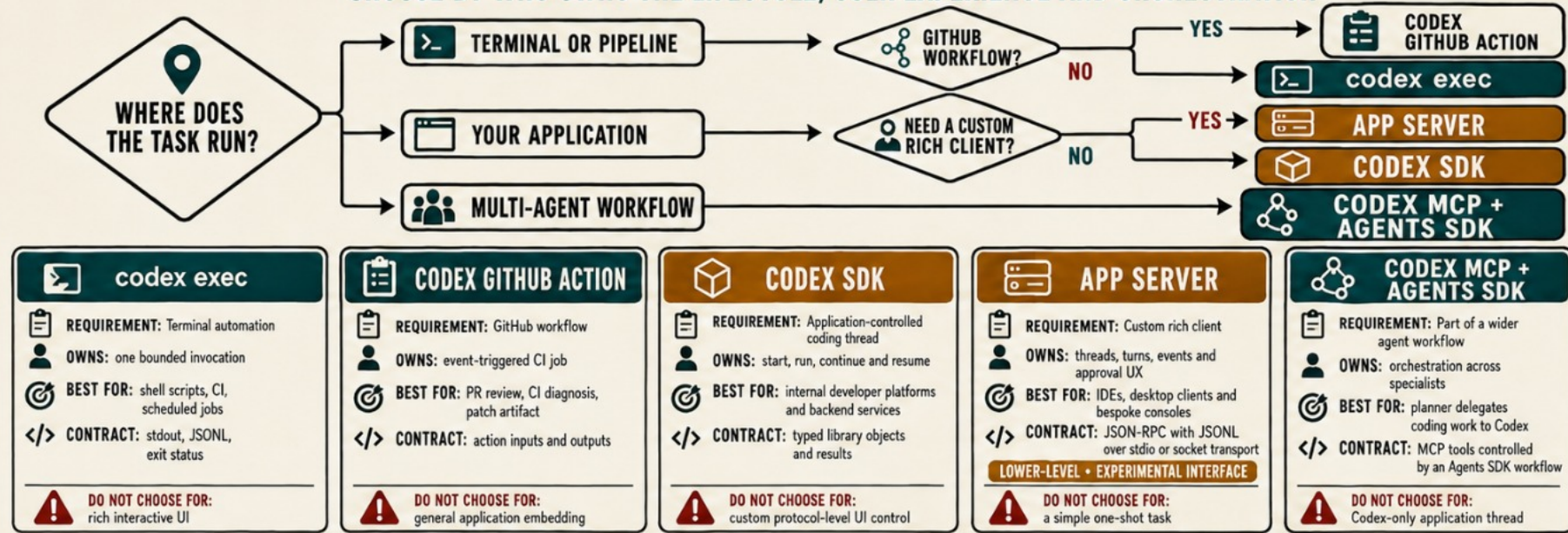
AN AUTOMATED REVIEWER MUST NEVER HOLD THE CREDENTIAL OR PERMISSION THAT DEPLOYS TO PRODUCTION

SAFE AUTOMATION = ISOLATED CREDENTIALS + REVIEWABLE ARTIFACTS + PROTECTED BRANCHES + HUMAN RELEASE AUTHORITY



MODULE 21 – SELECT THE CORRECT CODEX INTEGRATION

CHOOSE BY WHO OWNS THE LIFECYCLE, USER EXPERIENCE AND ORCHESTRATION

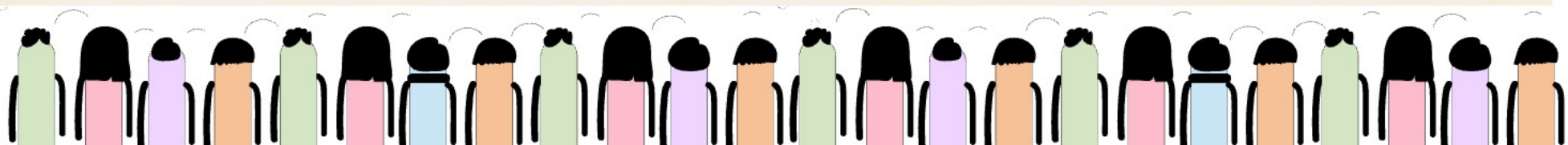


	codex exec	CODEX GITHUB ACTION	CODEX SDK	APP SERVER	CODEX MCP + AGENTS SDK
👤 LIFECYCLE OWNER	Shell	GitHub	Application	Client UI	Orchestrator
💬 INTERACTION	One-shot	Event job	Thread API	Streaming protocol	Tool delegation
📄 TYPICAL OUTPUT	Final result	Review artifact	Typed result	Live events	Agent handoff

⚠️ DO NOT USE APP SERVER WHEN THE SDK ALREADY PROVIDES THE THREAD CONTROL YOU NEED

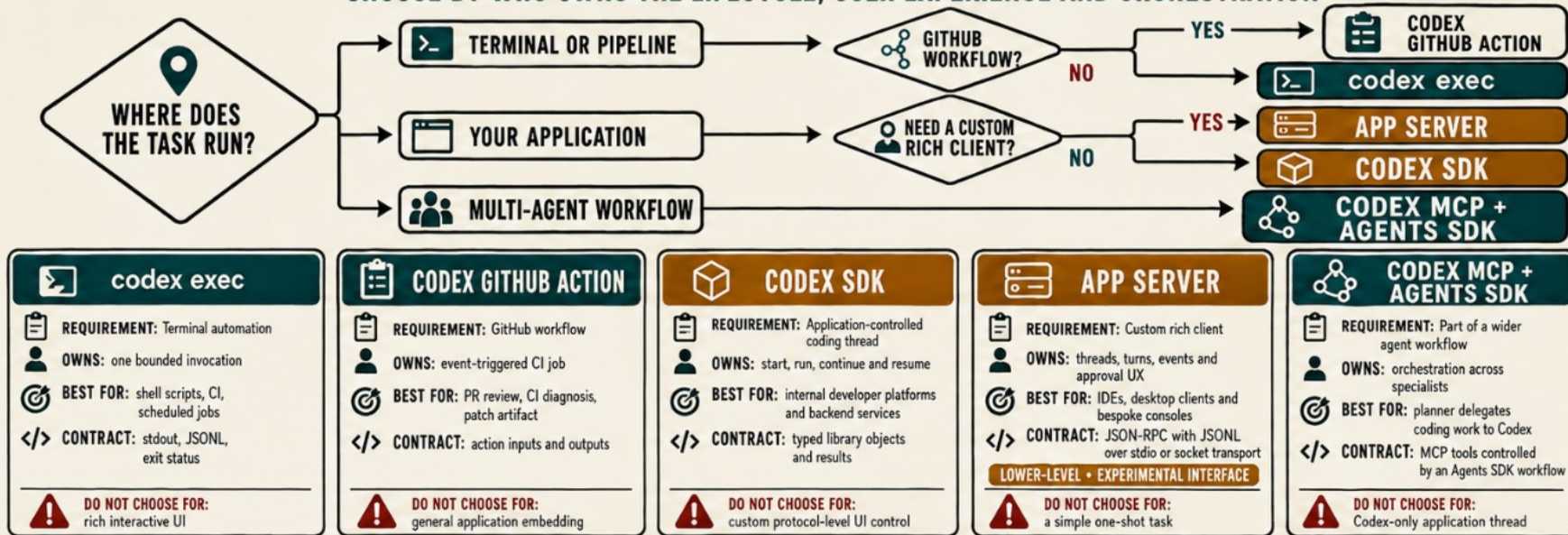
💡 REAL SCENARIO: Internal engineering portal starts and resumes coding work → **CODEX SDK**; custom IDE renders live plan, diff and approvals → **APP SERVER**

✅ CORRECT INTERFACE = EXECUTION LOCATION + LIFECYCLE OWNER + REQUIRED CONTROL



MODULE 21 – SELECT THE CORRECT CODEX INTEGRATION

CHOOSE BY WHO OWNS THE LIFECYCLE, USER EXPERIENCE AND ORCHESTRATION

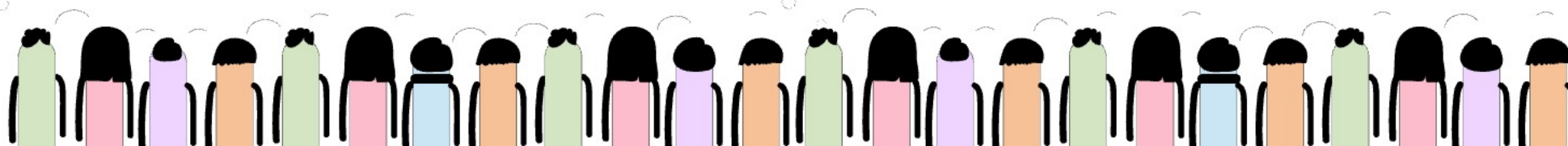


	codex exec	CODEX GITHUB ACTION	CODEX SDK	APP SERVER	CODEX MCP + AGENTS SDK
👤 LIFECYCLE OWNER	Shell	GitHub	Application	Client UI	Orchestrator
💬 INTERACTION	One-shot	Event job	Thread API	Streaming protocol	Tool delegation
📄 TYPICAL OUTPUT	Final result	Review artifact	Typed result	Live events	Agent handoff

⚠️ DO NOT USE APP SERVER WHEN THE SDK ALREADY PROVIDES THE THREAD CONTROL YOU NEED

💡 REAL SCENARIO: Internal engineering portal starts and resumes coding work → CODEX SDK; custom IDE renders live plan, diff and approvals → APP SERVER

✅ CORRECT INTERFACE = EXECUTION LOCATION + LIFECYCLE OWNER + REQUIRED CONTROL



CODEX SDK — APPLICATION-CONTROLLED CODING THREADS

SERVER-SIDE TYPESCRIPT • PERSISTENT THREAD IDENTITY • SAFE PROGRAMMATIC CONTROL

1 — SET UP

Node.js 18+
npm install @openai/codex-sdk
import { Codex } from "@openai/codex-sdk"
const codex = new Codex()

RUN SERVER-SIDE

2 — START

```
const thread = codex.startThread()
```

3 — RUN

```
await thread.run("Diagnose the CI failure")
```

4 — CONTINUE

```
await thread.run("Implement the smallest fix")
```

5 — STORE ID

Persist thread.id with application job record

6 — RESUME LATER

```
const thread2 = codex.resumeThread(threadId)  
await thread2.run("Continue validation")
```

Repeated
run() uses
the same
conversation
context.

resumeThread(
threadId)
reloads a
past thread.

THREAD CONFIGURATION

- working directory:** exact trusted repository
- sandbox:** read-only for investigation
- sandbox:** workspace-write for implementation
- approval policy:** match human availability
- model and reasoning effort:** task-based defaults
- environment:** no production secrets

CONFIGURE THE BOUNDARY BEFORE THE FIRST RUN

STRUCTURED RESULTS

- ✓ result.finalResponse for final text
- { } **JSON Schema** for stable downstream fields
- 🔍 Validate before database or workflow updates
- 📦 **Store:** thread ID, result, timestamps, commit SHA
- ⚠️ **Never parse informal prose for control decisions**

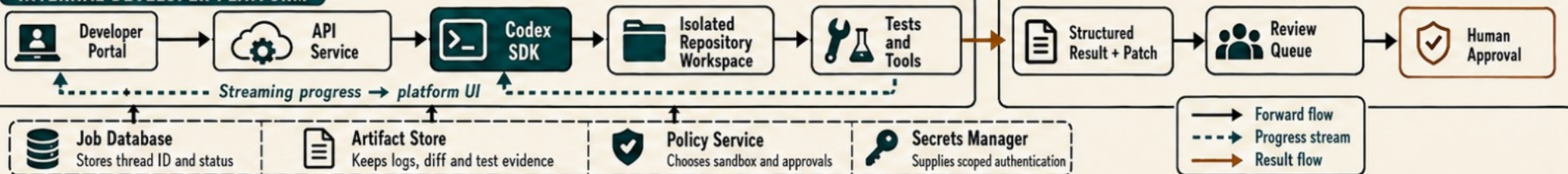
ERROR HANDLING

- ⚠️ Catch SDK and runtime errors
- 🔍 Classify transient transport vs task failure
- 🕒 Set caller timeout and cancellation
- 🔄 Bound retries with backoff
- 📦 Persist last known state
- ⚠️ **Never retry write operations blindly**

LONG-RUNNING TASK CONTROL

- 🔄 Use streaming progress when the UI needs status
- ☰ Show current phase: inspect, plan, edit, validate
- 🛑 Allow user cancellation
- 📦 Checkpoint thread ID and artifacts
- 🔄 Resume after worker restart
- ⏱️ Limit time, tokens and iterations
- ✅ End with evidence, not only a narrative

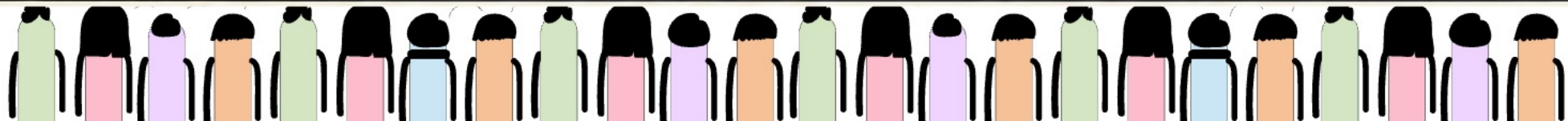
INTERNAL DEVELOPER PLATFORM



⚠️ **THE SDK CONTROLS A CODEX THREAD — IT DOES NOT REPLACE YOUR AUTHORIZATION, POLICY OR AUDIT LAYER**

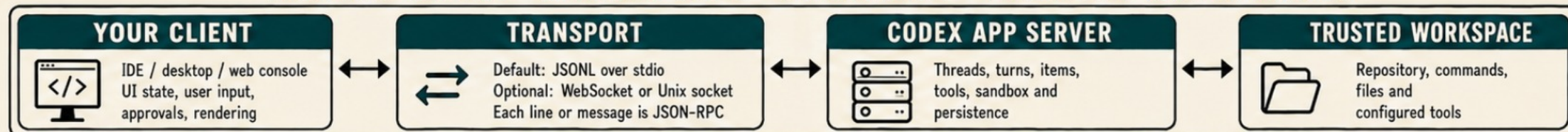
💡 **REAL SCENARIO:** Developer requests dependency upgrade → platform starts workspace-write thread → streams progress → stores thread ID → resumes after restart → returns patch and tests for review

✅ **APPLICATION OWNS POLICY AND UX • SDK OWNS THE CODING THREAD**

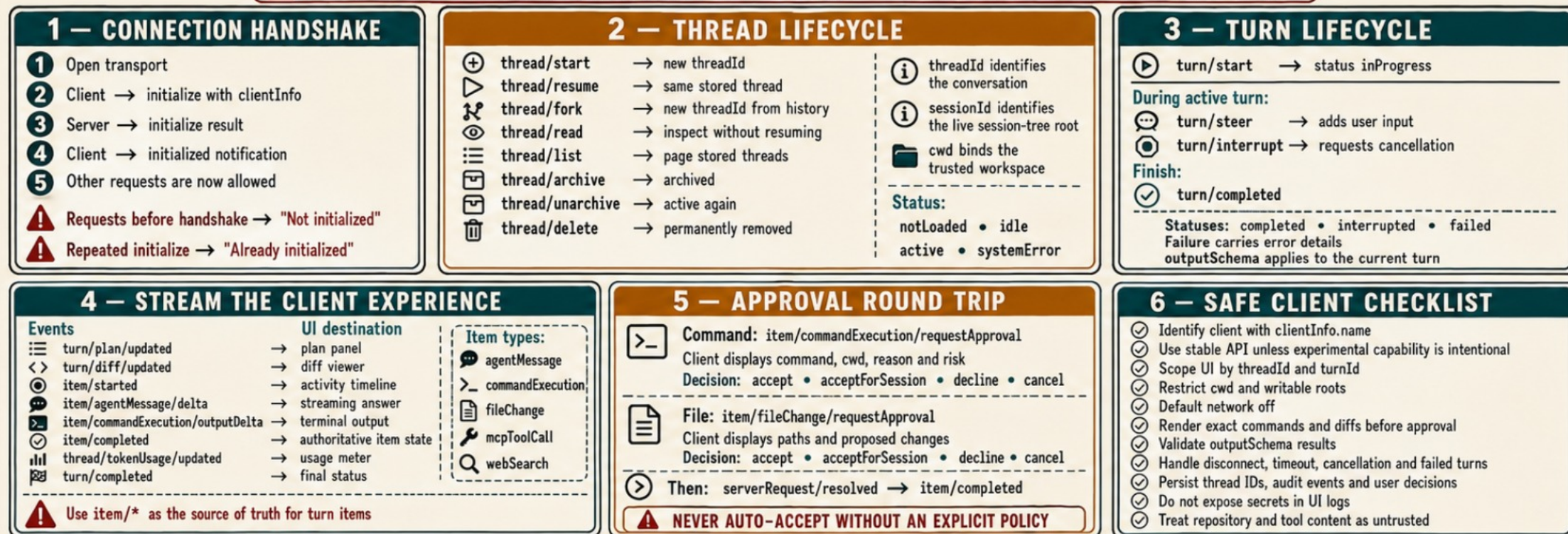


CODEX APP SERVER — BUILDING A SAFE RICH CLIENT

JSON-RPC CONTROL • STREAMED STATE • APPROVAL UX • THREAD AND TURN LIFECYCLES



⚠ APP SERVER IS A LOWER-LEVEL EXPERIMENTAL INTERFACE — PIN VERSIONS AND HANDLE PROTOCOL CHANGES



💡 REAL SCENARIO: Custom IDE initializes → starts thread in repository → starts turn → streams plan, command and diff → asks for file approval → user accepts → tests run → turn completes → client stores thread ID and audit trail

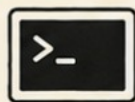
⚠ THE CLIENT MUST NOT CONFUSE A STREAMED DELTA WITH FINAL STATE — item/completed AND turn/completed ARE AUTHORITATIVE

🛡 SAFE RICH CLIENT = PROTOCOL STATE MACHINE + LEAST PRIVILEGE + EXPLICIT APPROVALS + AUDITABLE EVENTS

MODULE 22 — A ZERO EXIT CODE IS NOT PROOF

EVALUATION TURNS AI ADOPTION FROM FEELING INTO ENGINEERING EVIDENCE

FALSE CONFIDENCE



- ❌ Command exits 0
- ❌ Build appears green
- ❌ Agent says "done"

BUT IS THE CHANGE CORRECT?

THE INCIDENT CHAIN



Hidden defect



Production release



Customer impact



Emergency rollback

EVIDENCE-BASED CONFIDENCE

- ✓ Known task
- ✓ Objective graders
- ✓ Repeatable score
- ✓ Human calibration
- ✓ Release decision



“ A COMMAND EXITING ZERO DOES NOT PROVE THE GENERATED CODE IS CORRECT. ”

1 — EXECUTION QUALITY

- ✓ 1 Build success
- 🧪 2 Focused test pass rate
- 📊 3 Full regression pass rate
- 🔧 4 API compatibility
- 📈 5 Performance regression

2 — CHANGE QUALITY

- 🎯 6 Patch correctness
- 🎯 7 Scope adherence
- 📄 8 Diff size
- 🔄 9 Rollback readiness

3 — RISK + TRUTH

- 🔒 10 Security findings
- ❓ 11 Unsupported claims
- 🚫 12 Code-review false positives

4 — HUMAN OUTCOME

- 👥 13 Human acceptance rate
- 👤 Reviewer confidence (output)
- ✓ Production-readiness decision (output)

TASK SPECIFICATION

- 📋 Intent
- 📋 Constraints
- 📋 Acceptance criteria
- 📋 Test plan

AGENT RUN

- 🤖 Plan
- 🤖 Act
- 👁️ Observe
- 🔄 Iterate

PATCH AND TRACE

- 📄 Code diff
- 📄 Artifacts
- 📄 Logs
- 📄 Decisions

AUTOMATED GRADERS

- ⚙️ Compiler
- ⚙️ Tests
- ⚙️ Static analysis
- ⚙️ Security scan
- 📄 API contract
- 📄 Benchmark
- 📄 Diff policy

HUMAN REVIEW

- 👤 Intent
- 👤 Maintainability
- 👤 Unsupported claims
- 👤 False positives
- 👤 Acceptance

SCORECARD



RELEASE GATE

- 🚧 Pass: release
- 🚧 Fail: block
- 🚧 Rework or escalate

Metric	Threshold	Observed	Evidence	Pass or Fail



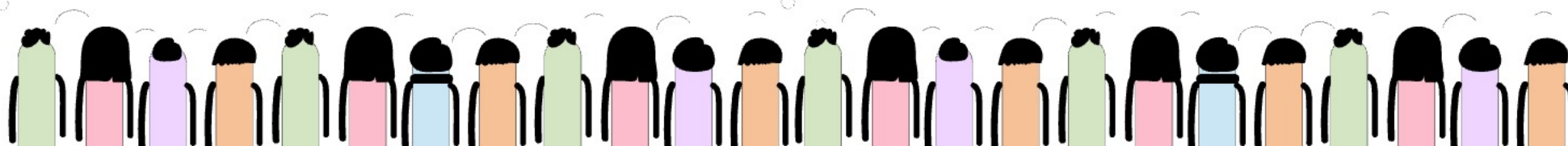
NO SINGLE METRIC PROVES QUALITY — BUILD, BEHAVIOR, RISK AND HUMAN ACCEPTANCE MUST AGREE



REAL SCENARIO: Agent fixes payment retry bug → build passes → focused test passes → regression reveals duplicate charge → evaluation blocks release



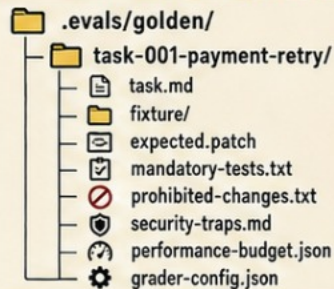
IF QUALITY CANNOT BE MEASURED, IT CANNOT BE GOVERNED



GOLDEN-TASK EVALUATION SET

VERSIONED, REPEATABLE PROOF FOR EVERY MODEL, PROMPT AND INSTRUCTION CHANGE

REPOSITORY STRUCTURE



ANATOMY OF ONE GOLDEN TASK

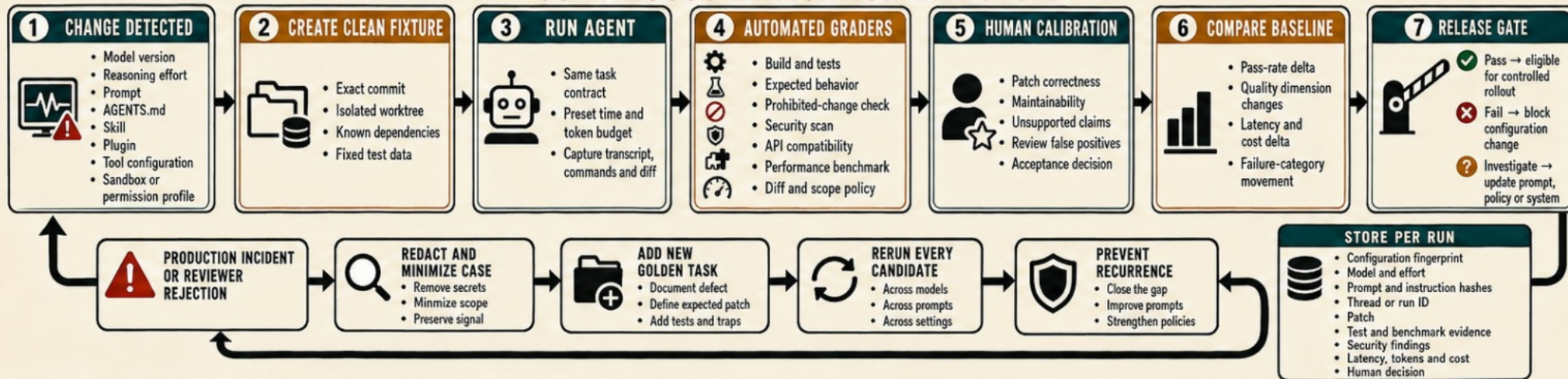
- Known defect** — exact failure intentionally present
- Expected patch** — reference behavior, not necessarily only one textual diff
- Mandatory tests** — must pass
- Prohibited changes** — files, APIs or behavior that must remain untouched
- Security traps** — secrets, injection, unsafe commands or vulnerable patterns
- Performance constraints** — latency, throughput, memory or query budget
- Acceptance rubric** — correctness, scope, evidence and rollback

TASK DESIGN RULES

- ✓ Representative of real engineering work
- ✓ Includes normal, edge and adversarial cases
- ✓ Deterministic setup and reset
- ✓ Independent expected evidence
- ✓ No hidden production credentials
- ✓ Reviewed by domain experts
- ✓ Versioned with the repository
- ✓ Hard enough to expose weak workflows



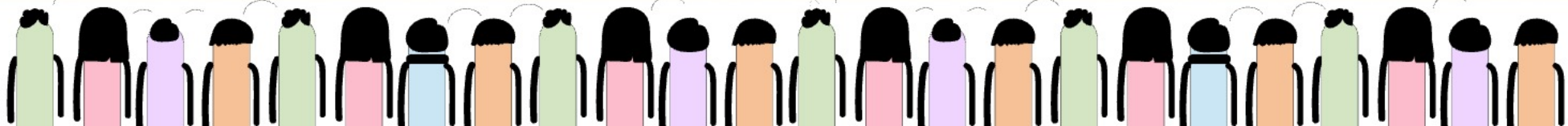
CONTINUOUS EVALUATION LIFECYCLE



⚠ **NEVER CHANGE THE TASK, GRADER AND CANDIDATE AT THE SAME TIME — YOU WILL NOT KNOW WHAT CAUSED THE RESULT**

★ **REAL SCENARIO:** New model produces a broader payment patch → mandatory tests pass → prohibited-change grader detects an auth-file edit → rollout blocked

🛡 **GOLDEN TASK = CONTROLLED INPUT + KNOWN CONSTRAINTS + OBJECTIVE EVIDENCE + REPEATABLE DECISION**



CONTROLLED COMPARISONS — CHANGE ONE VARIABLE

THE SAME GOLDEN TASK, THE SAME EVIDENCE AND A NEUTRAL SCORECARD

KEEP CONSTANT

- Exact repository commit
- Identical golden task
- Same tests and graders
- Same sandbox and tool access
- Same time and token budget
- Same starting state
- Same number of repeated runs
- Same human-review rubric

=

CHANGE ONE VARIABLE

- Prompt quality
- Repository instructions
- Permission mode
- Agent topology
- Reasoning effort
- Workflow structure
- Coding assistant

1 TASK DESCRIPTION

A	VS	B
 Weak, vague request		 Structured objective, constraints, evidence and acceptance criteria

2 REPOSITORY GUIDANCE

A	VS	B
 No repository instructions		 Applicable AGENTS.md standards

3 OPERATING MODE

A	VS	B
 Read-only review		 Workspace-write implementation

Compare review quality and implementation quality separately

4 AGENT TOPOLOGY

A	VS	B
 Single agent		 Main agent plus bounded specialist review

5 REASONING EFFORT

A	VS	B
 Lower effort		 Higher effort

Measure quality, latency and cost together

6 WORKFLOW

A	VS	B
 Unstructured prompting		 Skill-enabled repeatable workflow

7 ASSISTANT

A	VS	B
 Claude		 Codex

IDENTICAL TASK • IDENTICAL ENVIRONMENT • NO PREFERENCE

RUN PROTOCOL

- 1  Randomize run order
- 2  Reset clean fixture
- 3  Run multiple trials per variant
- 4  Capture patch, transcript, tools and timing
- 5  Apply blind or identity-hidden grading where possible
- 6  Compare confidence intervals, not one lucky run
- 7  Review failure categories, not only averages

NEUTRAL SCORECARD (RUBRIC)					
CRITERION	VARIANT A	VARIANT B	DELTA	EVIDENCE	DECISION
 Build		 Performance			
 Focused tests		 Diff size			
 Regression		 Unsupported claims			
 Patch correctness		 False positives			
 Scope		 Human acceptance			
 Security		 Rollback readiness			
 API compatibility					

DECISION GUIDE (USE EVIDENCE, NOT HYPE)	
	Higher quality + acceptable cost → adopt
	Similar quality + lower cost or latency → prefer efficient option
	Mixed result → task-specific routing
	Lower quality or unsafe → reject
	Noisy evidence → run more trials



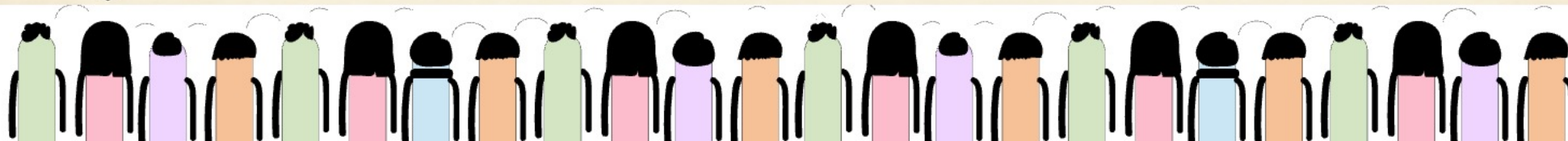
DO NOT CHANGE THE MODEL, PROMPT, TOOLS AND PERMISSIONS TOGETHER — THAT IS A DEMO, NOT AN EXPERIMENT



REAL SCENARIO: Claude and Codex run the identical payment golden task five times → hidden graders score patches → team selects by evidence per task category



AN HONEST COMPARISON MEASURES CAPABILITY — IT DOES NOT DEFEND A PREFERENCE



CONTROLLED COMPARISONS — CHANGE ONE VARIABLE

THE SAME GOLDEN TASK, THE SAME EVIDENCE AND A NEUTRAL SCORECARD

KEEP CONSTANT

- Exact repository commit
- Identical golden task
- Same tests and graders
- Same sandbox and tool access
- Same time and token budget
- Same starting state
- Same number of repeated runs
- Same human-review rubric

CHANGE ONE VARIABLE

- Prompt quality
- Repository instructions
- Permission mode
- Agent topology
- Reasoning effort
- Workflow structure
- Coding assistant

1 TASK DESCRIPTION

A	VS	B
Weak, vague request		Structured objective, constraints, evidence and acceptance criteria

2 REPOSITORY GUIDANCE

A	VS	B
No repository instructions		Applicable AGENTS.md standards

3 OPERATING MODE

A	VS	B
Read-only review		Workspace-write implementation

Compare review quality and implementation quality separately

4 AGENT TOPOLOGY

A	VS	B
Single agent		Main agent plus bounded specialist review

5 REASONING EFFORT

A	VS	B
Lower effort		Higher effort

Measure quality, latency and cost together

6 WORKFLOW

A	VS	B
Unstructured prompting		Skill-enabled repeatable workflow

7 ASSISTANT

A	VS	B
Claude		Codex

IDENTICAL TASK • IDENTICAL ENVIRONMENT • NO PREFERENCE

RUN PROTOCOL

- Randomize run order
- Reset clean fixture
- Run multiple trials per variant
- Capture patch, transcript, tools and timing
- Apply blind or identity-hidden grading where possible
- Compare confidence intervals, not one lucky run
- Review failure categories, not only averages

NEUTRAL SCORECARD (RUBRIC)					
CRITERION	VARIANT A	VARIANT B	DELTA	EVIDENCE	DECISION
Build		Performance			
Focused tests		Diff size			
Regression		Unsupported claims			
Patch correctness		False positives			
Scope		Human acceptance			
Security		Rollback readiness			
API compatibility					

DECISION GUIDE (USE EVIDENCE, NOT HYPE)	
✓	Higher quality + acceptable cost → adopt
\$	Similar quality + lower cost or latency → prefer efficient option
⚡	Mixed result → task-specific routing
✗	Lower quality or unsafe → reject
?	Noisy evidence → run more trials



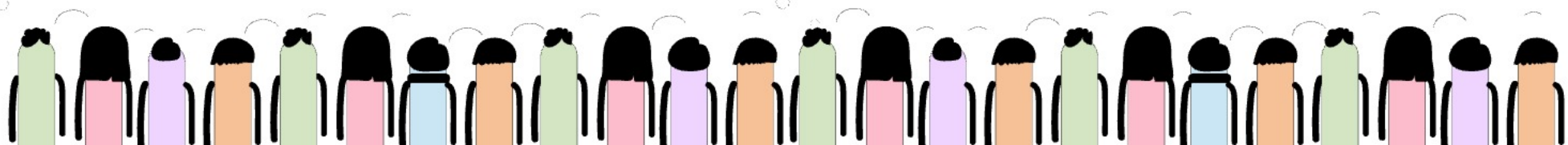
DO NOT CHANGE THE MODEL, PROMPT, TOOLS AND PERMISSIONS TOGETHER — THAT IS A DEMO, NOT AN EXPERIMENT



REAL SCENARIO: Claude and Codex run the identical payment golden task five times → hidden graders score patches → team selects by evidence per task category



AN HONEST COMPARISON MEASURES CAPABILITY — IT DOES NOT DEFEND A PREFERENCE



CODEX OBSERVABILITY — FROM AGENT EVENT TO OPERATING SIGNAL

OPEN TELEMETRY • REDACTION • LATENCY • APPROVALS • COST • DASHBOARDS

1 — ENABLE OTel IN config.toml

```
[otel]
environment = "staging"
exporter = "none"
log_user_prompt = false
```

- Use otlp-http or otlp-grpc to export
- exporter = "none" sends nothing
- OTel export is disabled by default
- Prompts remain redacted unless explicitly enabled
- Use an environment variable for exporter credentials

🛡️ DEFAULT TO METADATA, NOT CONTENT

2 — CORE CODEX EVENTS

💬	codex.conversation_starts	model, reasoning, sandbox, approval policy
☁️	codex.api_request	attempt, status, success, duration, error
(:o:)	codex.sse_event	kind, success, duration, token counts
👤	codex.user_prompt	length; content redacted by default
🔧	codex.tool_decision	approved or denied; config or user
🔧	codex.tool_result	tool, duration, success, safe output snippet

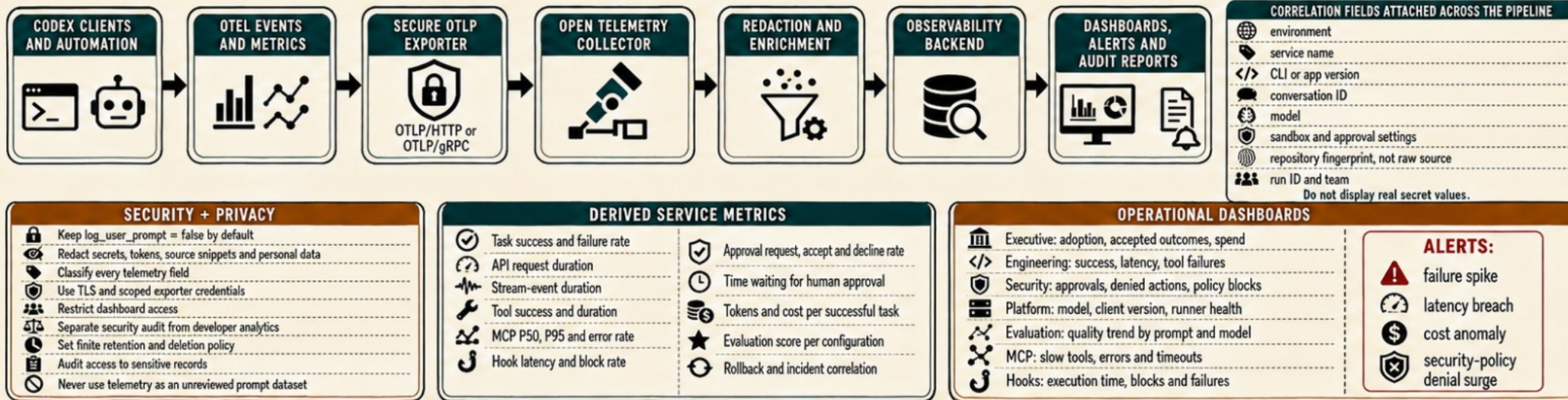
3 — ENGINEERING ACTIVITY

💬	Conversation start
🔧	Command and file-tool execution
🛡️	Approval request and decision
🔗	MCP tool call duration and result
👤	Hook started, completed, blocked or failed
⚠️	Turn failure and cancellation
🕒	End-to-end task latency
📊	Token usage and estimated cost

NOTE:



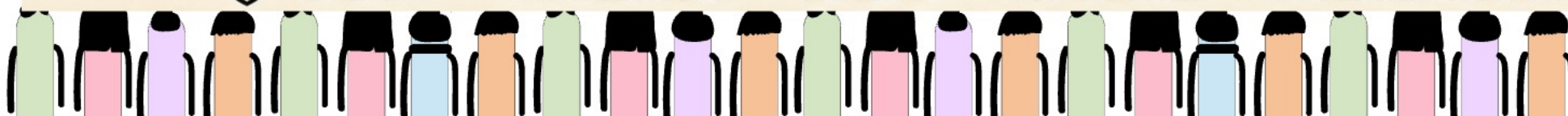
Use tool events and application spans to correlate MCP calls and hook execution



⚠️ **OBSERVABILITY WITHOUT REDACTION BECOMES A DATA-LEAK PIPELINE**

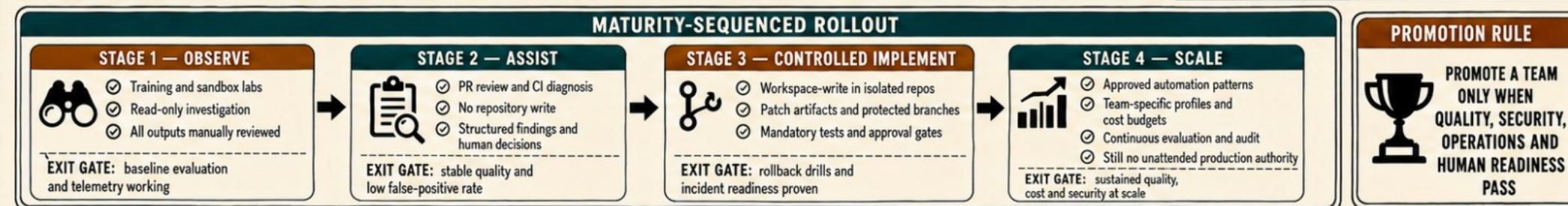
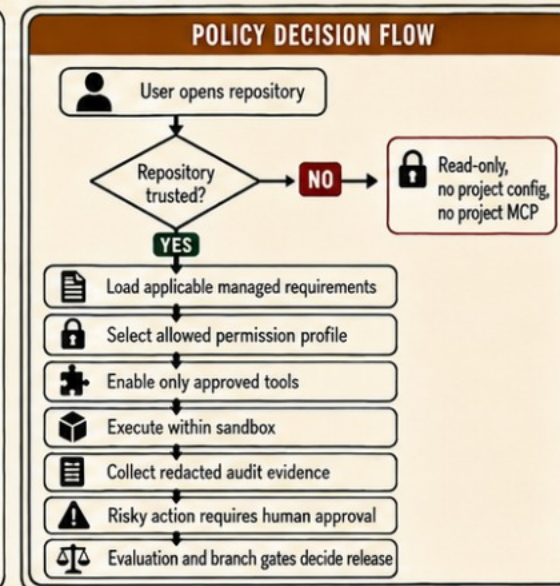
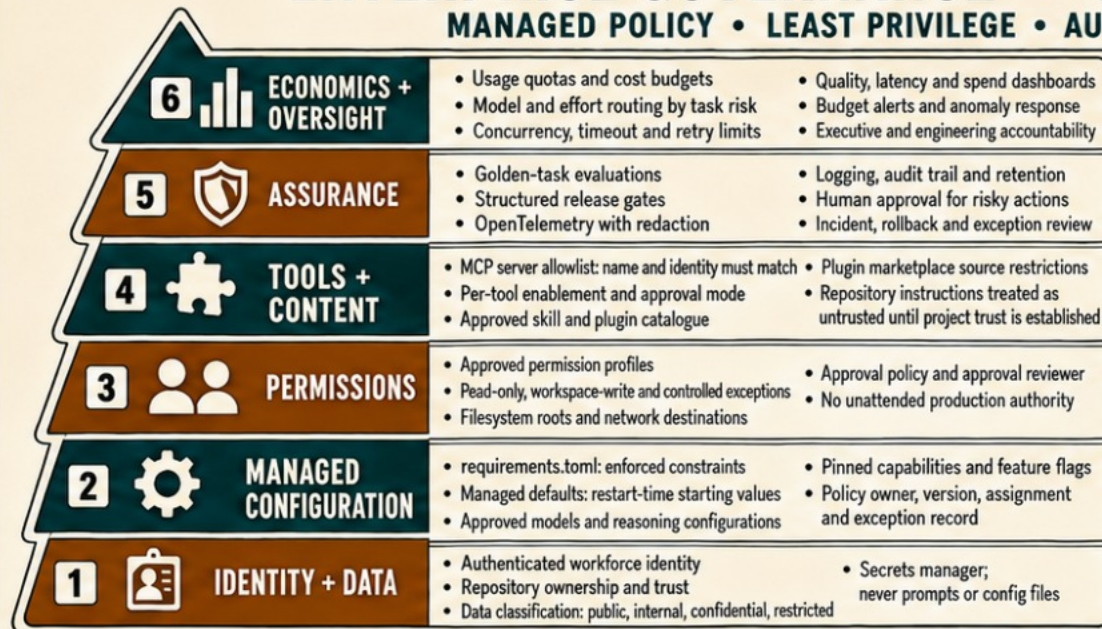
★ **REAL SCENARIO:** MCP issue-tracker latency rises → tool-duration dashboard shows P95 breach → alert identifies one server → platform disables it without exposing prompts

🛡️ **EVENTS → CORRELATION → METRICS → DASHBOARD → GOVERNED ACTION**



ENTERPRISE GOVERNANCE — AUTONOMY MUST BE EARNED

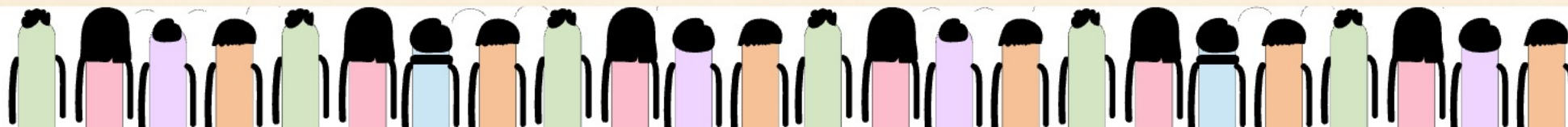
MANAGED POLICY • LEAST PRIVILEGE • AUDITABLE USE • MATURITY-BASED ROLLOUT

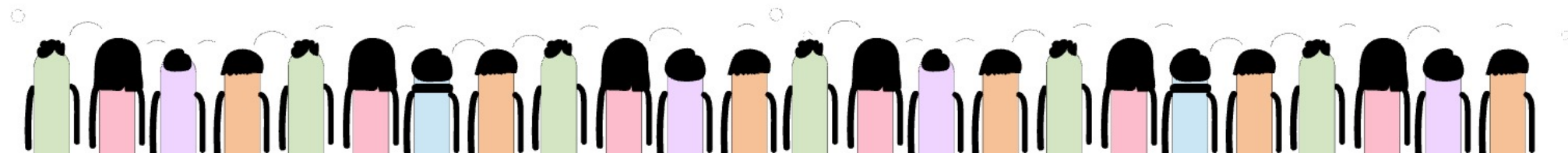


⚠️ **MANAGED CONFIGURATION REDUCES RISK — IT DOES NOT REPLACE RBAC, ENVIRONMENT ISOLATION OR HUMAN ACCOUNTABILITY** ⚠️

★ **REAL SCENARIO:** Platform team pilots read-only PR review → golden-task and false-positive targets pass → controlled patch workflow enabled → production deployment remains separately human-approved ★

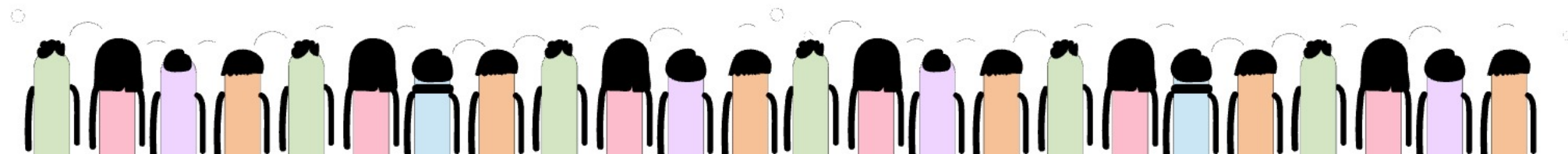
🛡️ **GOVERNANCE = POLICY + TECHNICAL ENFORCEMENT + EVIDENCE + ACCOUNTABLE HUMANS** 🛡️



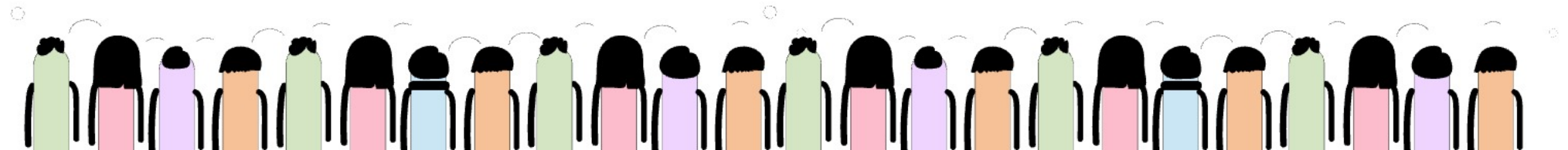
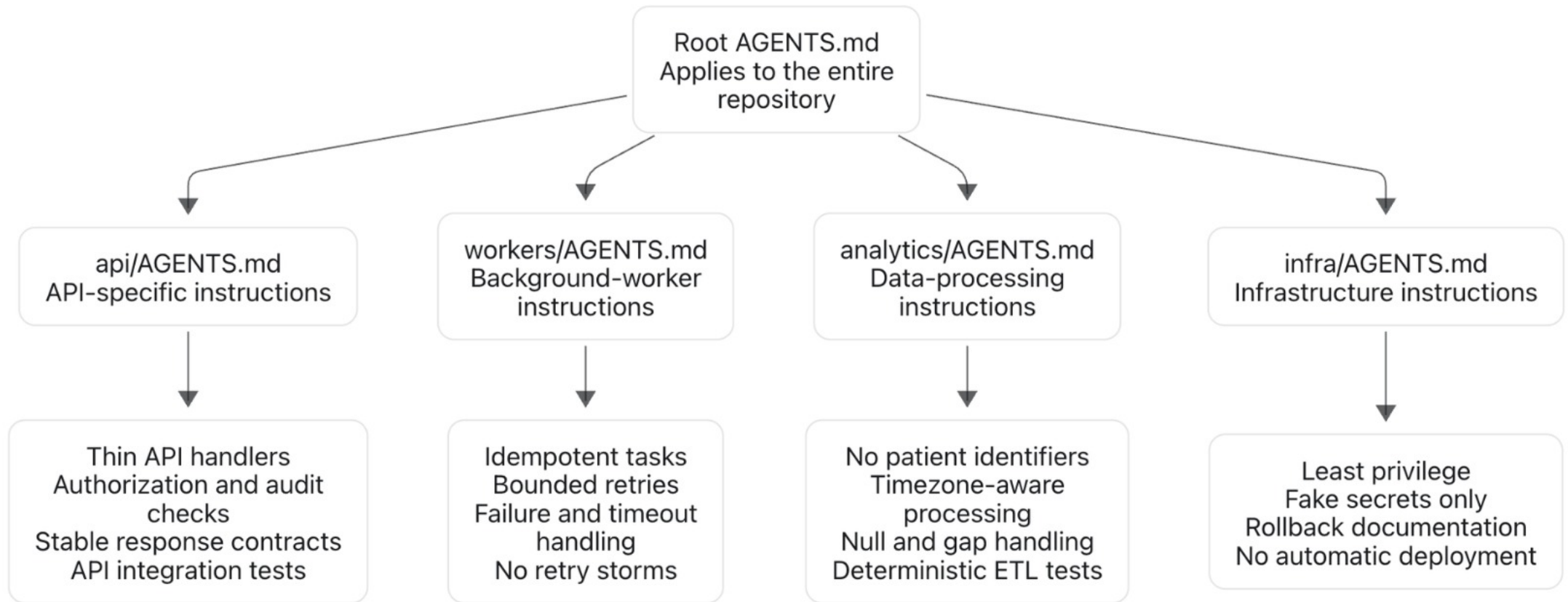




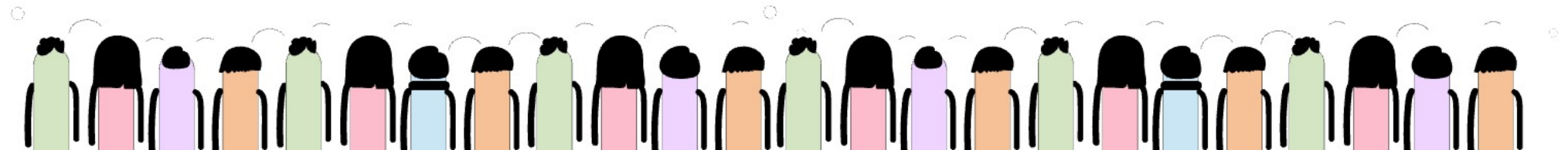
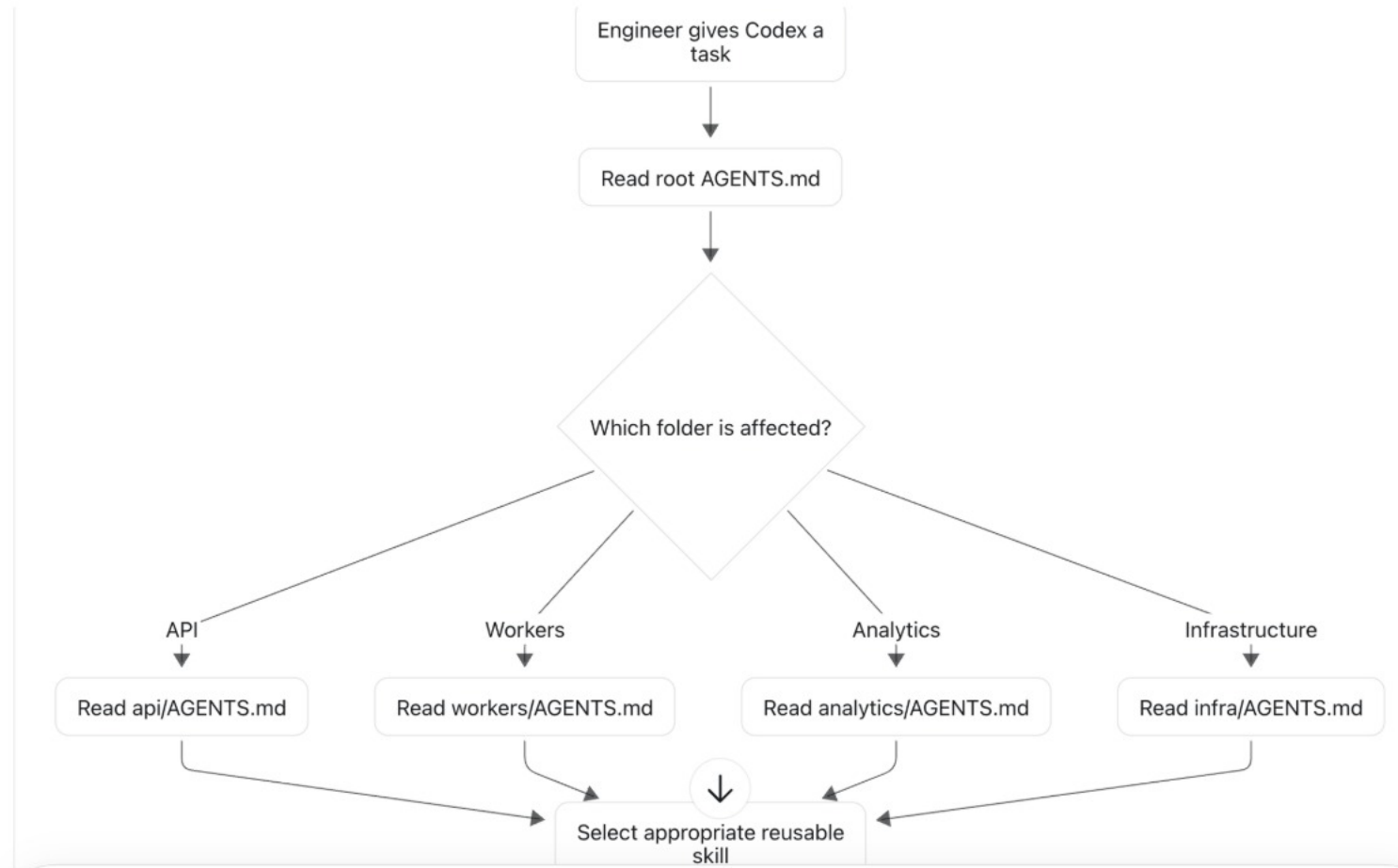
Codex



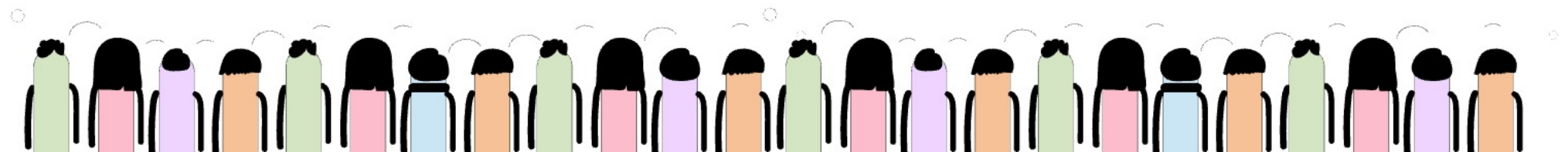
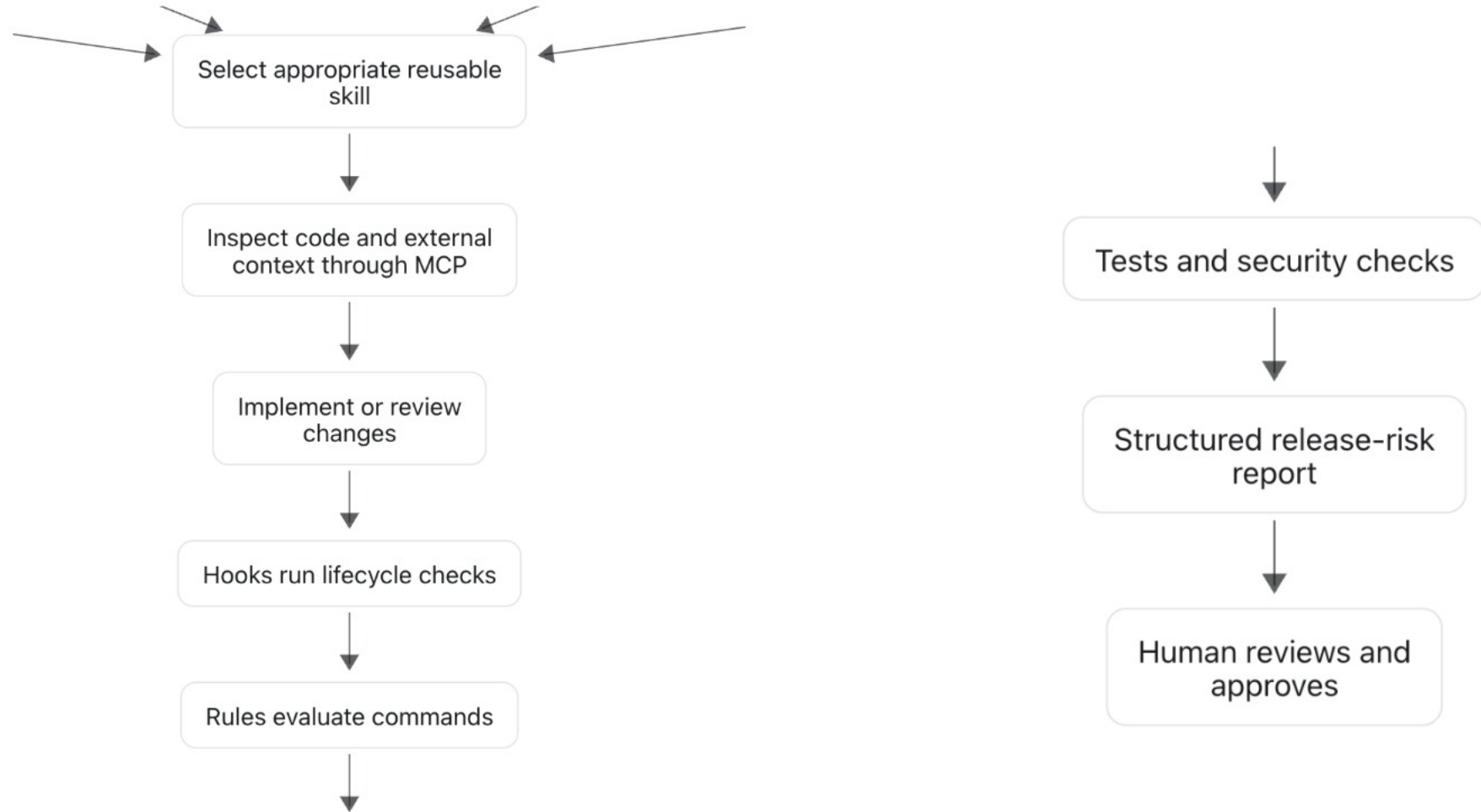
Agent



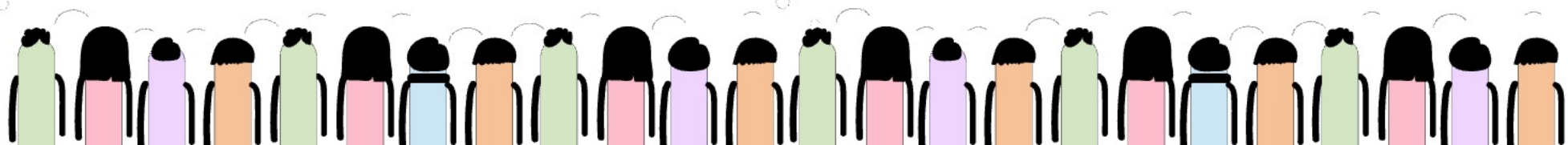
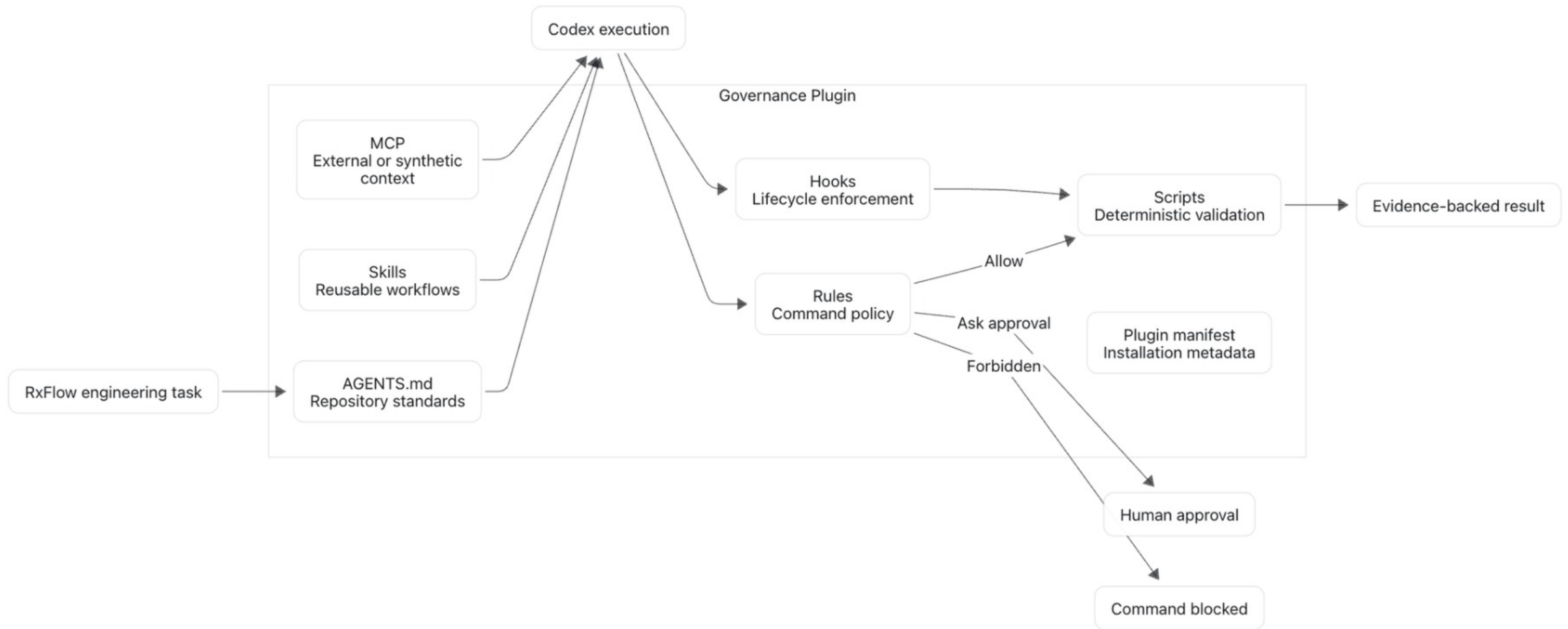
Agent

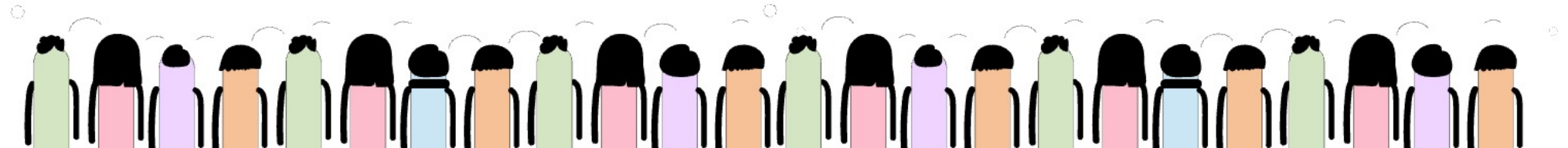
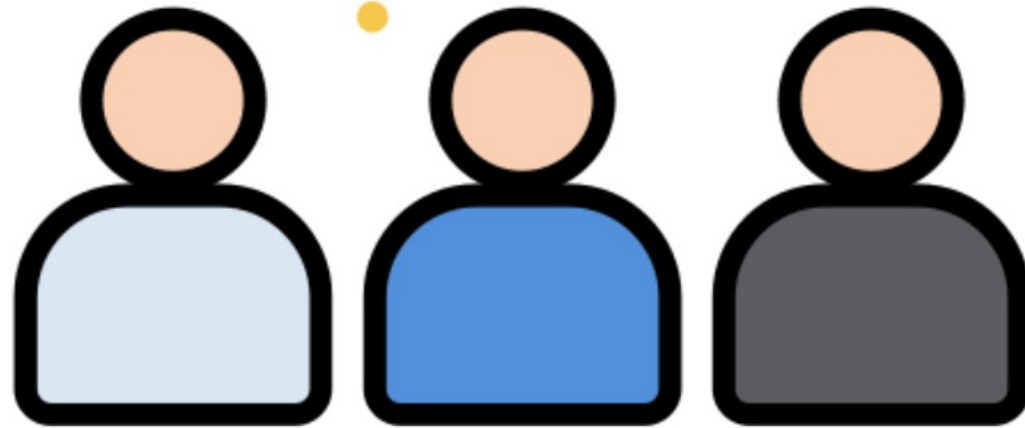


Agent



Governance







TESTIMONIALS

